



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Python para Ciencia de Datos

Tema 11

Visualización de datos con matplotlib y seaborn

Máster F.P. En IA & Big Data ANALYTICS
Curso 2026-2027

UPV

Índice



- 1. Introducción**
- 2. Matplotlib y pandas**
- 3. Seaborn**

Resultados de aprendizaje

- Conocer el pipeline de datos habitual en Ciencia de Datos para analizar y manipular los datasets
- Comprender el uso de Series y DataFrames
- Operar con DataFrames de forma correcta y óptima
- Analizar y manipular estructuras de datos complejas

Introducción

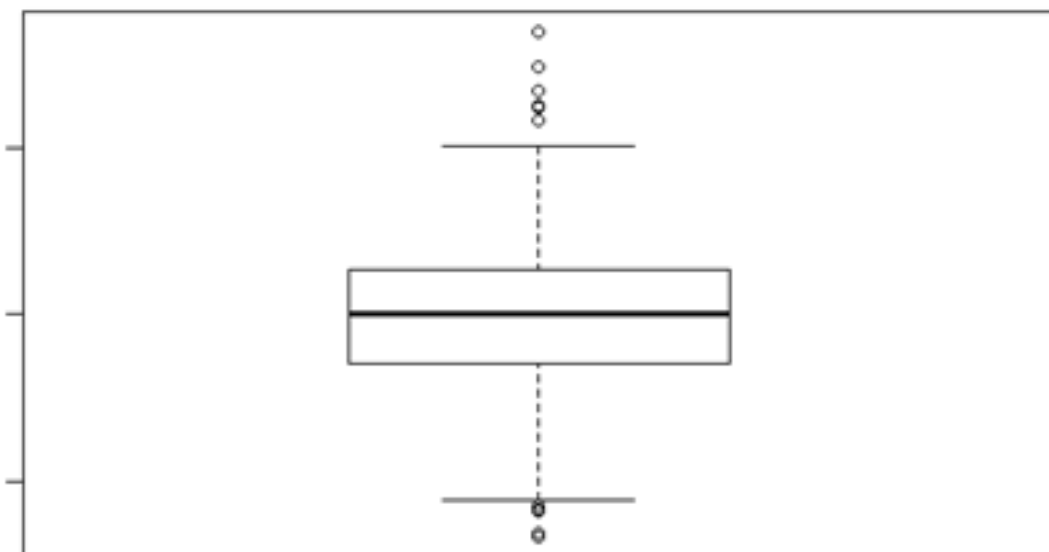
La visualización de datos

Variable	Tipo de Variable	Visualización
Univariantes (Una Variable)	Datos Continuos	▪ Histograma ▪ Box Plot (Diagrama de cajas) ▪ Diagrama de Violin ▪ Kernel Density Curve
	Datos Categóricos (nominales, ordinales)	▪ Diagrama de Tarta ▪ Diagrama de Barras
Multivariantes (Dos o Más Variables)	Datos Continuos	▪ Diagrama de dispersion (Scatter Plot) ▪ Diagrama de Líneas ▪ Diagrama de Serie Temporal
	Datos Categóricos (nominales, ordinales)	▪ Diagrama de Mosaico

Introducción

Tipos básicos de visualización

- Visualizaciones univariantes:
 - Una variable numérica continua → **Boxplot**.



1.Caja (Box):

Representa el **rango intercuartílico (IQR)**: los valores entre el **1er cuartil (Q1, 25%)** y el **3er cuartil (Q3, 75%)**.

Dentro de la caja está el **50% central de los datos**.

2.Línea dentro de la caja (Median line):

Es la **mediana (Q2, 50%)** de los datos.

3.Bigotes (Whiskers):

Se extienden desde los bordes de la caja hasta el valor más extremo que no sea considerado un *outlier*.

Por convención, los bigotes suelen llegar hasta

$Q1 - 1.5 \cdot IQR$ y $Q3 + 1.5 \cdot IQR$.

4. Outliers (puntos sueltos):

Valores que quedan fuera del rango definido por los bigotes.

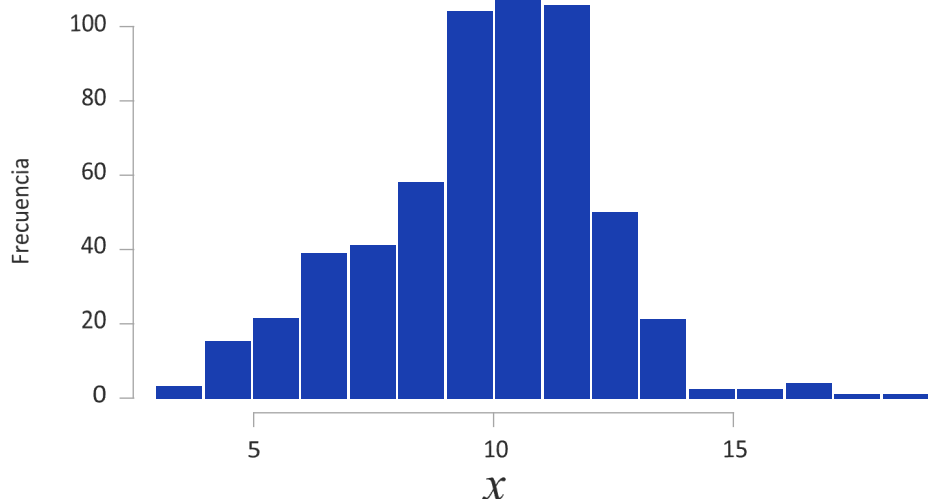
Se muestran como círculos o puntos individuales.

- Un Boxplot se compone de una **caja**, **“bigotes”** y **outliers**.

Introducción

Tipos básicos de visualización

- Visualizaciones univariantes:
 - Una variable numérica continua → **Histograma**.



1. Eje X (horizontal):

Representa los intervalos o clases en que se agrupan los datos (por ejemplo, rangos de edades, ingresos, etc.).

Cada intervalo tiene el mismo ancho (en la mayoría de los casos).

2. Eje Y (vertical):

Representa la frecuencia de los datos:

Frecuencia absoluta → número de elementos en cada intervalo.

Frecuencia relativa → proporción o porcentaje de elementos.

3. Barras (rectángulos):

La anchura de la barra = el ancho del intervalo.

La altura de la barra = la frecuencia correspondiente a ese intervalo.

A diferencia de un gráfico de barras categóricas, en el histograma las barras están pegadas porque representan un continuo.

4. Clases o Intervalos (Bins):

Los rangos en los que se dividen los datos continuos.

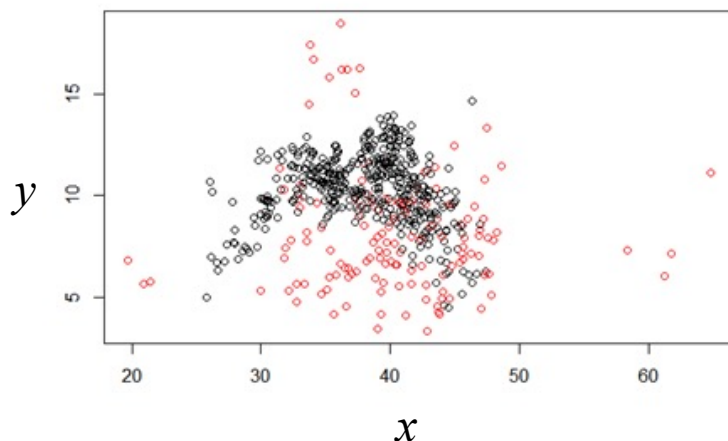
La elección del número de intervalos afecta la forma de la distribución.

- Muestra las frecuencias relativas o absolutas de cada intervalo

Introducción

Tipos básicos de visualización

- Visualizaciones multivariantes:
 - Dos variables numéricas continuas -> **Scatter Plot**.



1. Eje X (horizontal):

Representa la variable independiente o explicativa.

2. Eje Y (vertical):

Representa la variable dependiente o de respuesta.

3. Puntos (marcadores):

Cada punto corresponde a una observación (par ordenado (x, y)).

La posición indica el valor de cada variable para esa observación.

4. Tendencia o patrón (opcional):

Correlación positiva (sube en diagonal ↗).

Correlación negativa (baja en diagonal ↘).

Ausencia de correlación (puntos dispersos sin patrón claro).

5. Outliers (valores atípicos):

Puntos que se alejan significativamente del patrón general.

6. (Opcional) Variables adicionales:

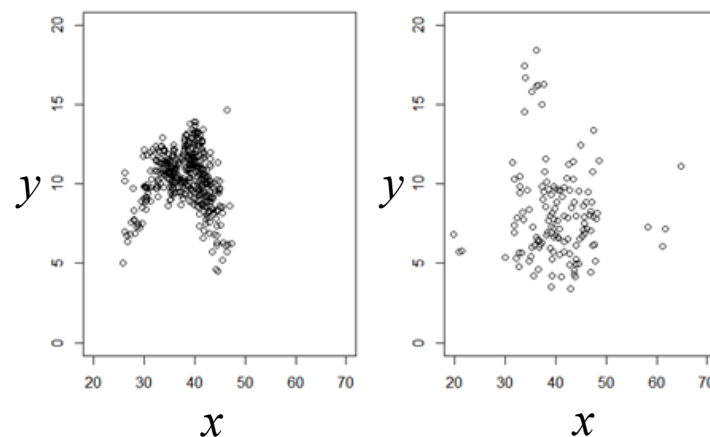
En visualizaciones enriquecidas, el color o el tamaño de los puntos puede representar una tercera variable.

- Permiten identificar una relación lineal entre dos variables

Introducción

Tipos básicos de visualización

- Visualizaciones multivariantes:
 - Dos variables numéricas continuas y una categórica -> **Multiple Scatter Plot**.

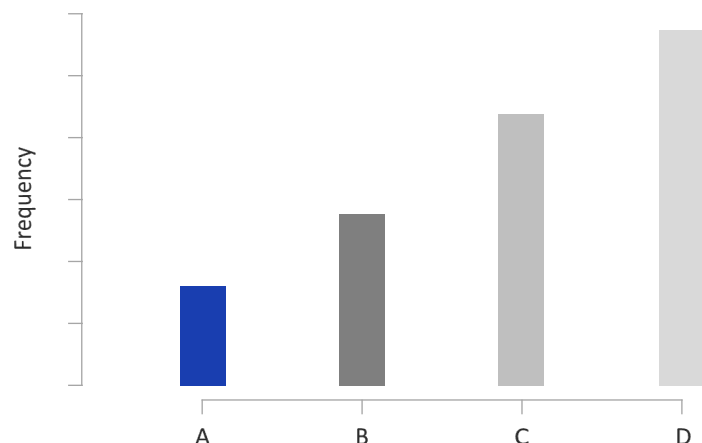


- **Diferentes categorías** se pueden dibujar por **separado**
- Asegúrate de que los ejes están en la **misma escala** para poder compararlos

Introducción

Tipos básicos de visualización

- Visualizaciones univariantes:
 - Una variable categórica -> **Bar Plot**.



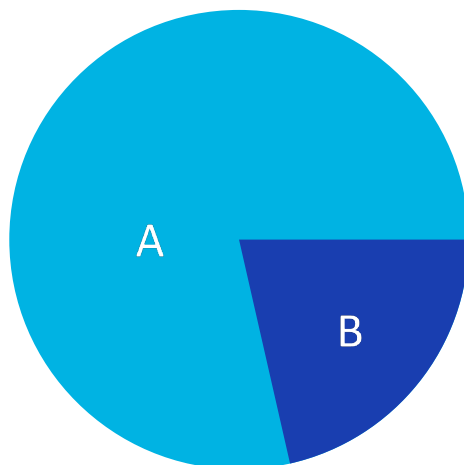
- 1. Eje X (horizontal, en un bar plot vertical):**
Representa las categorías o grupos de la variable cualitativa.
- 2. Eje Y (vertical):**
Representa la frecuencia, proporción o valor de cada categoría.
Puede ser frecuencia absoluta (número de casos) o relativa (porcentaje).
- 3. Barras:**
Cada barra corresponde a una categoría.
Altura de la barra = valor o frecuencia de la categoría.
Las barras suelen tener ancho constante y están separadas.
- 4. Etiquetas de categorías:**
Se colocan bajo cada barra (en eje X) o junto a ella si es un gráfico horizontal.

- Muestra la frecuencia relativa o absoluta de cada categoría

Introducción

Tipos básicos de visualización

- Visualizaciones multivariantes:
 - Una variable categórica -> **Gráfico de tarta (Pie Plot)**.



1. Porciones (sectores del círculo):

- Cada **porción** representa una **categoría** de la variable cualitativa.
- El **área** de la porción es proporcional a la frecuencia o **porcentaje** de la categoría.

2. Ángulos:

- El ángulo central de cada sector está en **proporción** al valor de la categoría.
- La suma de todos los ángulos es 360°.

3. Colores o patrones:

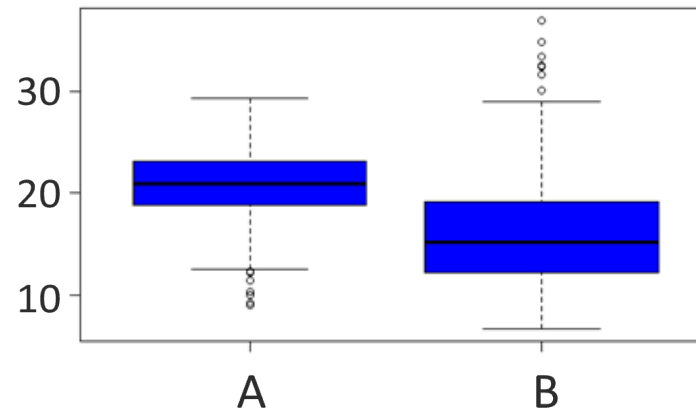
- Se utilizan para diferenciar cada categoría.
- Facilitan la **comparación visual** entre sectores.

- Muestra las proporciones de cada categoría

Introducción

Tipos básicos de visualización

- Visualizaciones bivariantes:
 - Una variable continua numérica y una categórica -> **Múltiples Box Plots.**

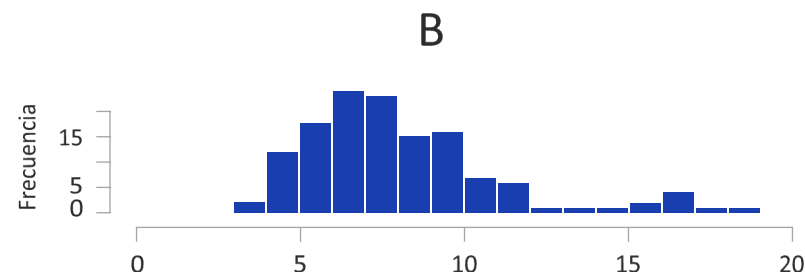
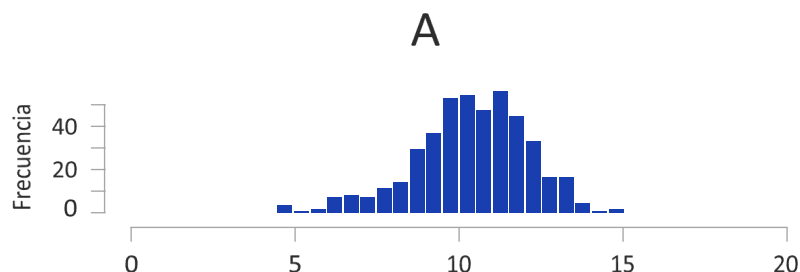


- El número de categorías es igual al número de boxplots.

Introducción

Tipos básicos de visualización

- Visualizaciones bivariantes:
 - Una variable continua numérica y una categórica → **Múltiples Histogramas.**

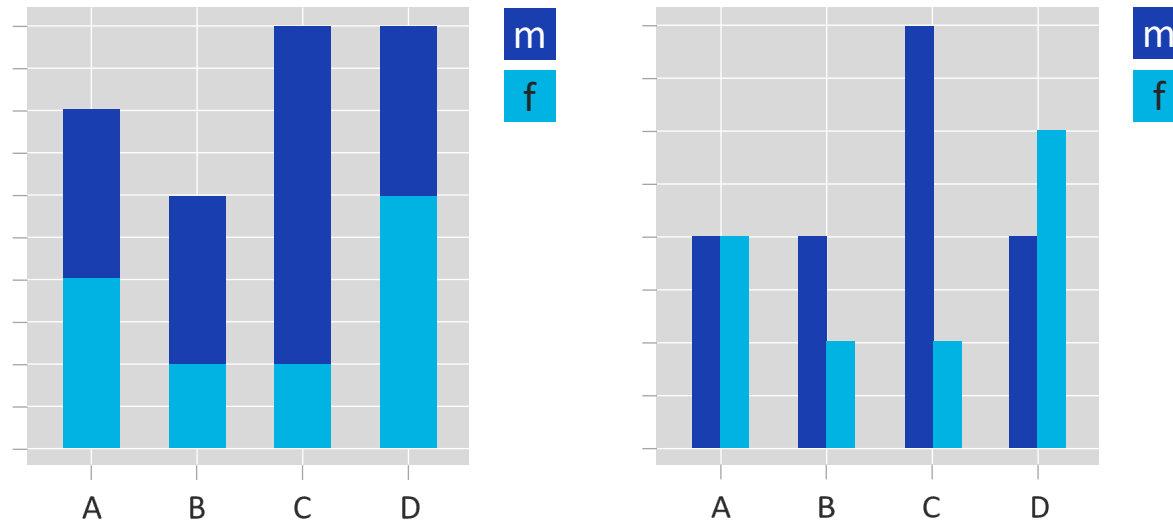


- El número de categorías es igual al número de histogramas.
- Asegúrate de que los rangos de los ejes coinciden para poder compararlos apropiadamente.

Introducción

Tipos básicos de visualización

- Visualizaciones bivariantes:
 - Dos variables categóricas -> **Gráfico de Barras (Bar Plot)**.



- Usa el color para distinguir las categorías de la variable secundaria.

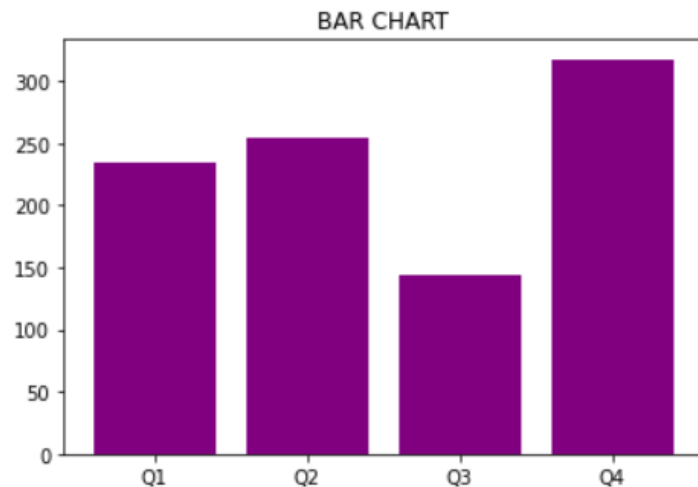
Visualización con matplotlib

Visualizaciones básicas

- Gráficos de Barras (Bar Plot)

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline

x = np.array(['Q1', 'Q2', 'Q3', 'Q4'])
y = np.array([234.0, 254.7, 144.6, 317.6])
plt.bar(x, y, color = 'purple')
plt.title('BAR CHART')
plt.show()
```

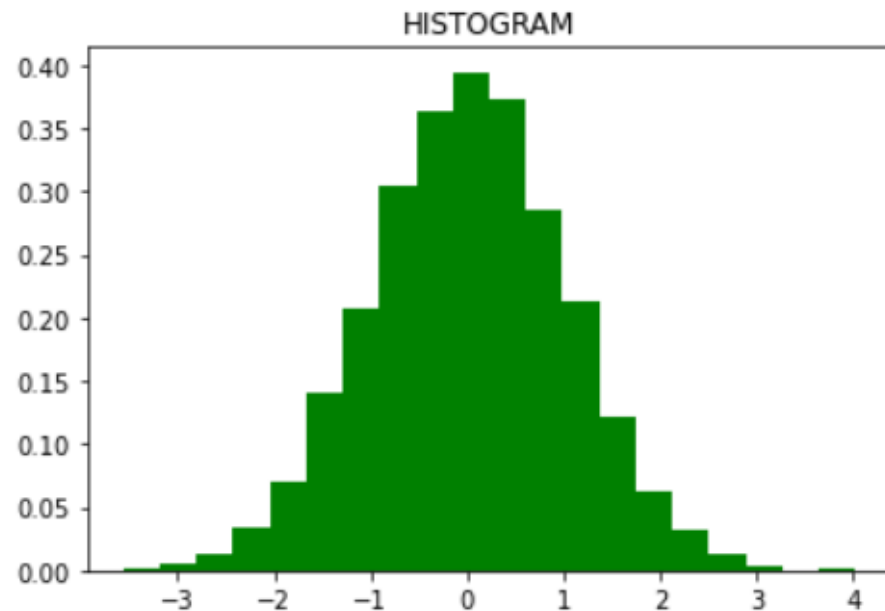


Visualización con matplotlib

Visualizaciones básicas

■ Histogramas

```
In [2]: x = np.random.randn(10000)
plt.hist(x, bins=20, color='green', density=True)
plt.title('HISTOGRAM')
plt.show()
```



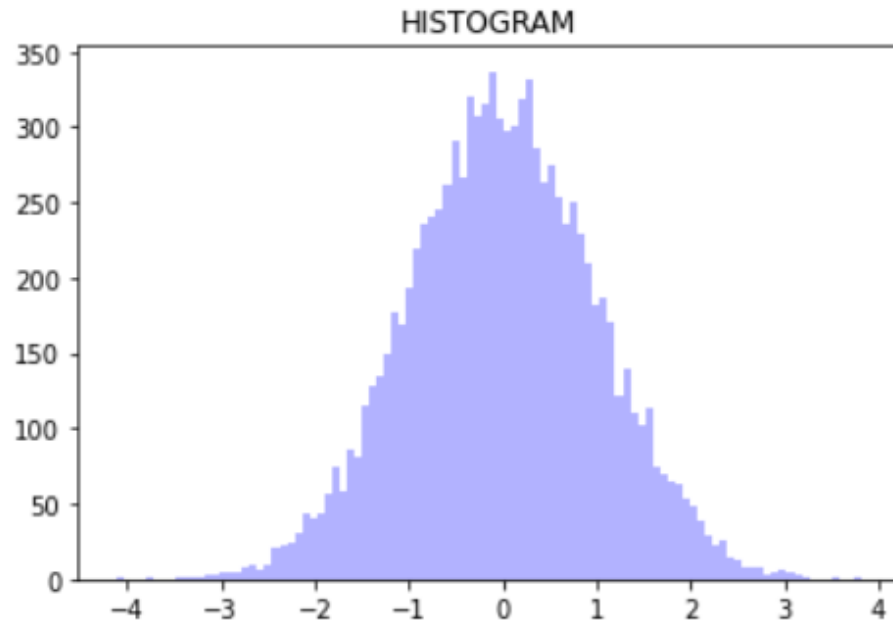
- **x**: Datos a representar
- **bins**: número de grupos
- **density=True**: porcentaje de datos en eje Y.

Visualización con matplotlib

Visualizaciones básicas

■ Histogramas

```
In [3]: x = np.random.randn(10000)
plt.hist(x, bins=100, color='blue', density=False, alpha=0.3)
plt.title('HISTOGRAM')
plt.show()
```



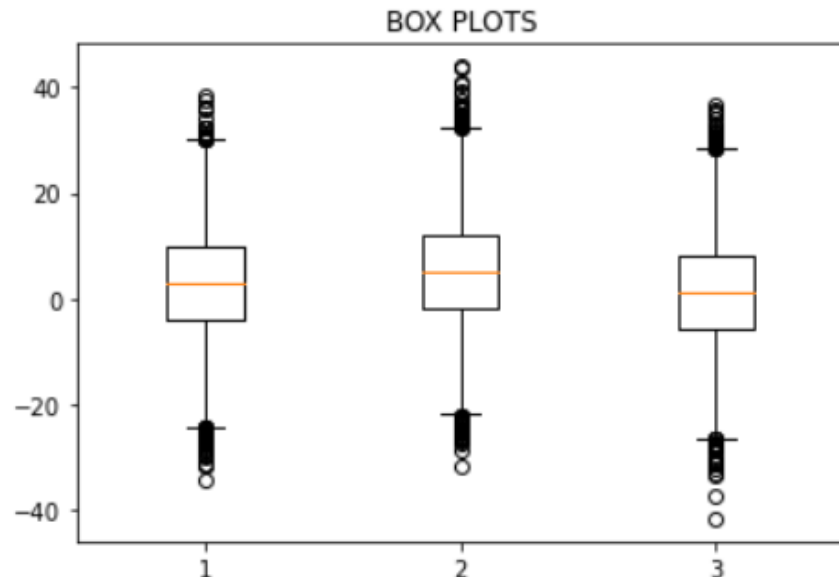
- **x**: Datos a representar
- **bins**: número de grupos
- **density=False**: cantidad de datos en eje Y.
- **alpha**: Transparencia (en porcentaje)

Visualización con matplotlib

Visualizaciones básicas

■ Box plots múltiples

```
In [4]: x = np.random.randn(10000)*10+3
y = np.random.randn(10000)*10+5
z = np.random.randn(10000)*10+1
plt.boxplot([x,y,z],0)
plt.title('BOX PLOTS')
plt.show()
```



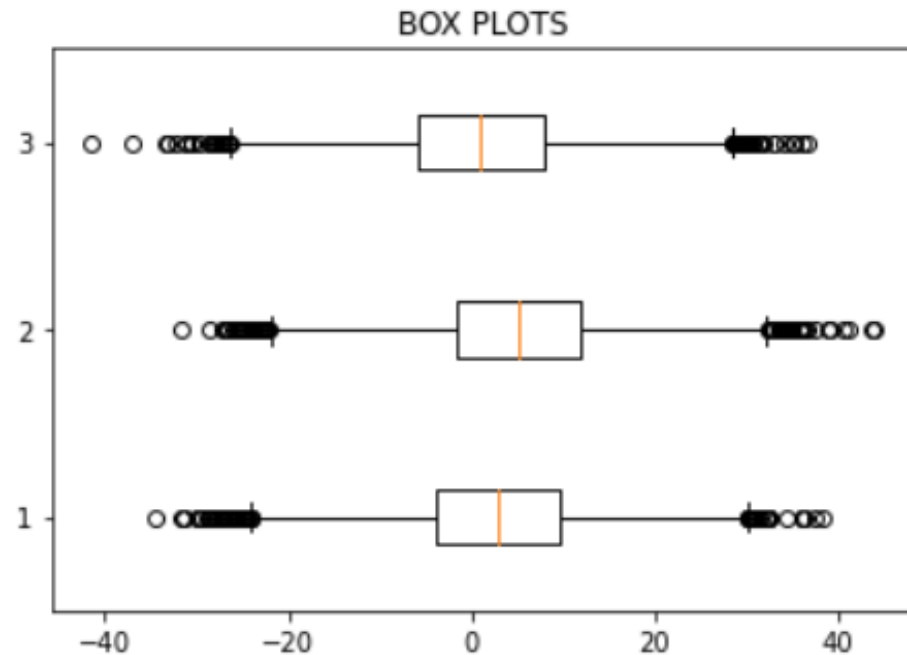
- **x, y, z:** Datos a representar
- **notch=0:** No representar las muescas (notchs) de intervalos de confianza.

Visualización con matplotlib

Visualizaciones básicas

■ Box plots múltiples

```
In [5]: plt.boxplot([x,y,z],0,vert=False)
plt.title('BOX PLOTS')
plt.show()
```



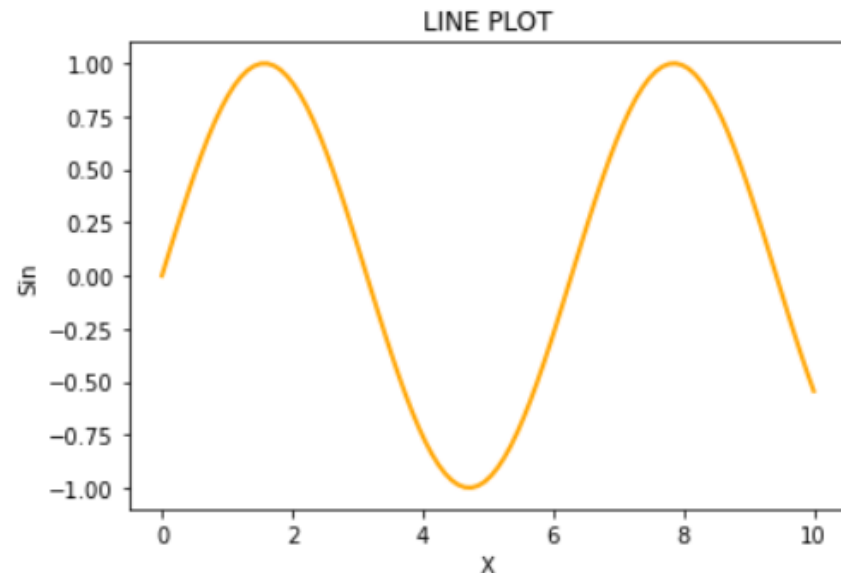
- **x, y, z:** Datos a representar
- **notch=0:** No representar las muescas (notchs) de intervalos de confianza.
- **vert=False:** Cajas en horizontal

Visualización con matplotlib

Visualizaciones básicas

■ Gráficos de líneas

```
In [6]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.plot(x,y,color='orange',linewidth=2)           # linestyle = '-'
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('LINE PLOT')
plt.show()
```



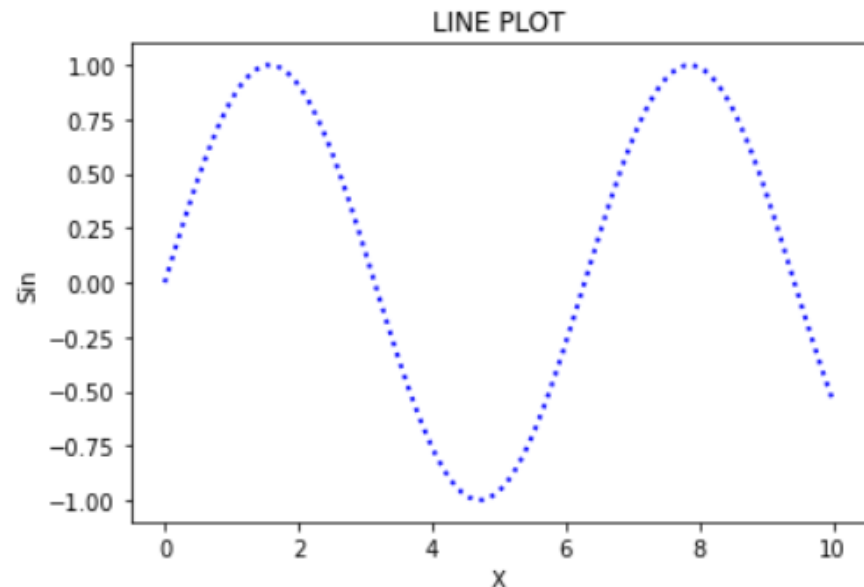
- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **linewidth**: Grosor de línea
- **linestyle**: Estilo de línea ("-", "--", "-.", ":", ...)

Visualización con matplotlib

Visualizaciones básicas

■ Gráficos de líneas

```
In [7]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.plot(x,y,color='blue',linestyle=':',linewidth=2)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('LINE PLOT')
plt.show()
```



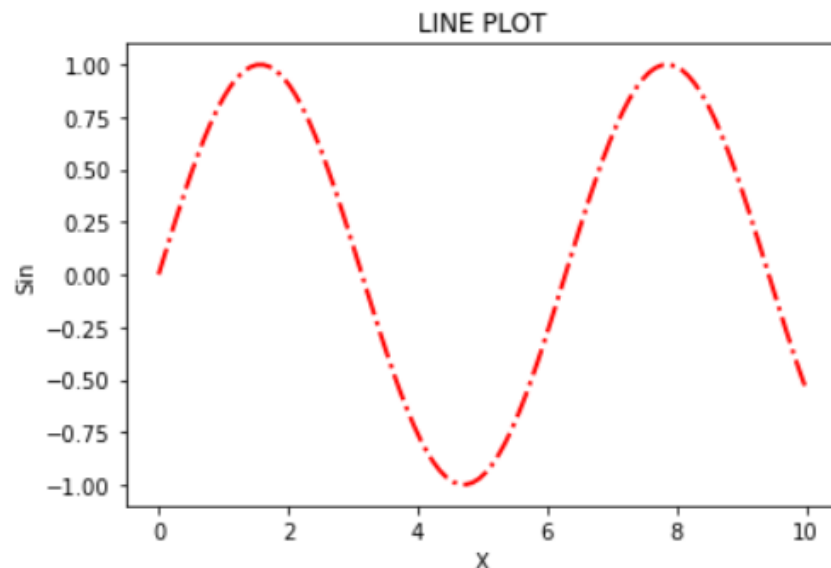
- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **linewidth**: Grosor de línea
- **linestyle**: Estilo de línea ("-", "--", "-.", ":", ...)

Visualización con matplotlib

Visualizaciones básicas

■ Gráficos de líneas

```
In [8]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.plot(x,y,color='red',linestyle='-.',linewidth=2)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('LINE PLOT')
plt.show()
```



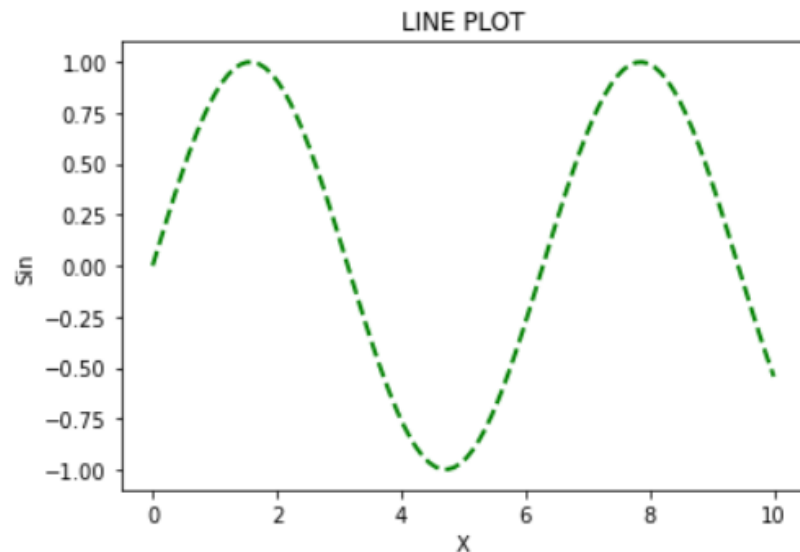
- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **linewidth**: Grosor de línea
- **linestyle**: Estilo de línea ("-", "--", "-.", ":", ...))

Visualización con matplotlib

Visualizaciones básicas

■ Gráficos de líneas

```
In [9]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.plot(x,y,color='green',linestyle='--',linewidth=2)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('LINE PLOT')
plt.show()
```



- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **linewidth**: Grosor de línea
- **linestyle**: Estilo de línea ("-", "--", "-.", ":", ...)

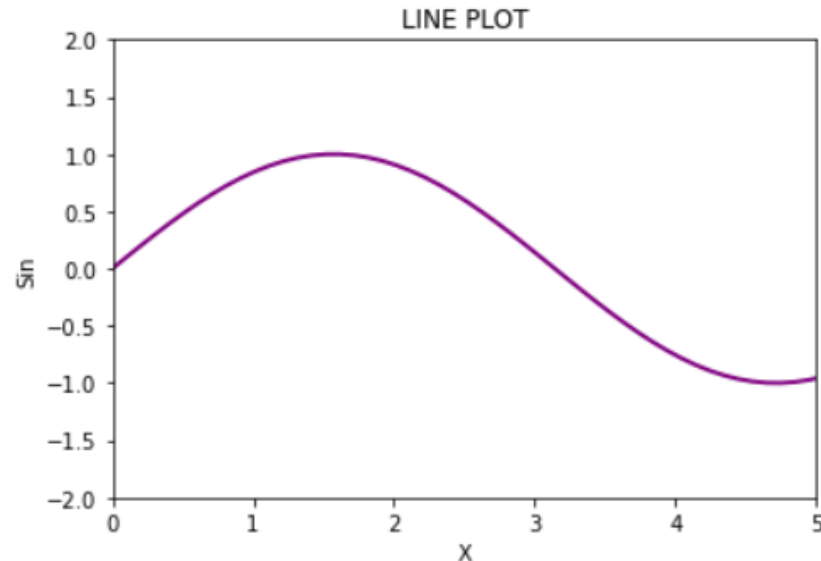
Visualización con matplotlib

Visualizaciones básicas

■ Gráficos de líneas

```
In [10]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.plot(x,y,color='purple',linestyle='-',linewidth=2)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('LINE PLOT')
plt.xlim([0,5])
plt.ylim([-2,+2])
plt.show()
```

Horizontal axis limits.
Vertical axis limits.

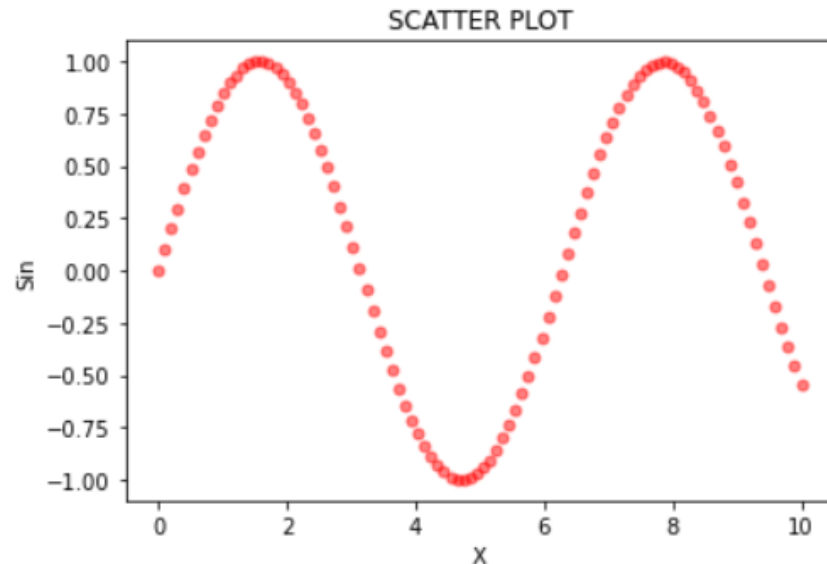


Visualización con matplotlib

Visualizaciones básicas

■ Scatter Plots

```
In [11]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.plot(x,y,color='red',marker='o',linestyle='none',markersize=5, alpha=0.5)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



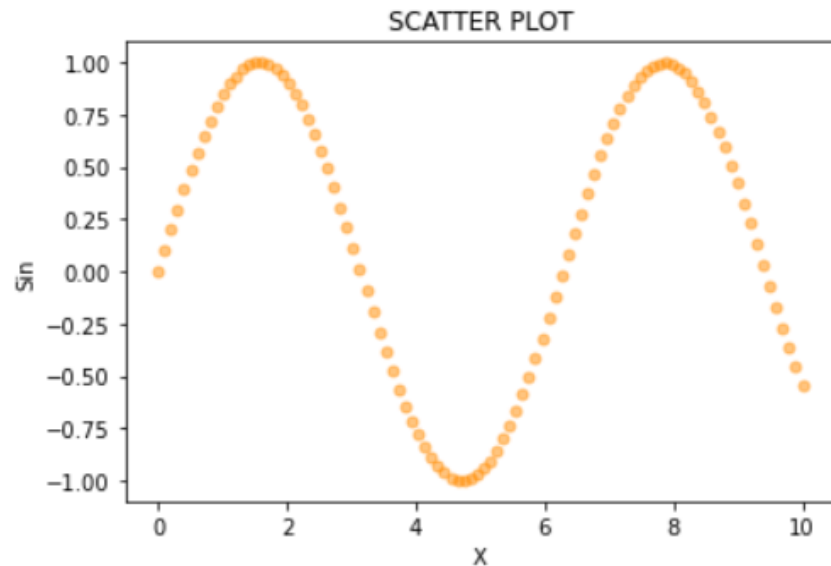
- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **marker**: Símbolo del marcador ("o", ".", "v", "^", "*", etc...)
- **markersize**: Tamaño del marcador
- **linestyle**: none.

Visualización con matplotlib

Visualizaciones básicas

■ Scatter Plots

```
In [12]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.plot(x,y,color='#FF8C00',marker='o',linestyle='none',markersize=5, alpha=0.5)    # RGB color.
plt.xlabel('x')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



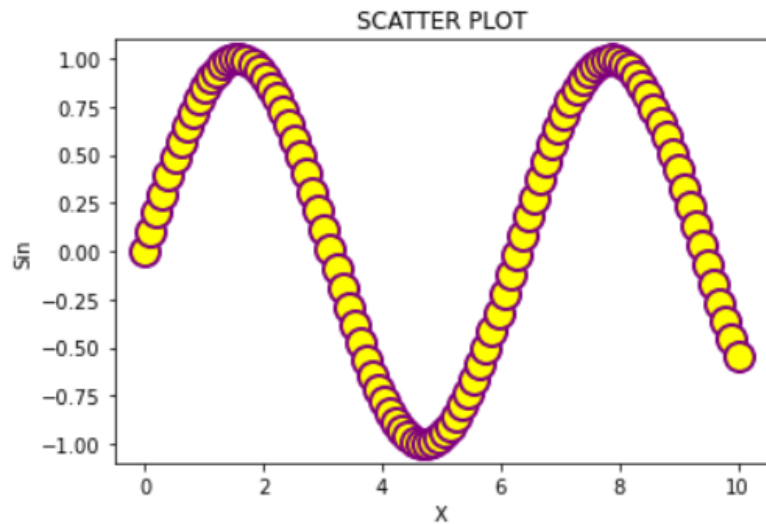
- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **marker**: Símbolo del marcador ("o", ".", "v", "^", "*", etc...)
- **markersize**: Tamaño del marcador
- **linestyle**: none.

Visualización con matplotlib

Visualizaciones básicas

■ Scatter Plots

```
In [13]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.plot(x,y,marker='o',linestyle='none',markersize=15,markerfacecolor='yellow',markeredgecolor='purple',markeredgewidth=2)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



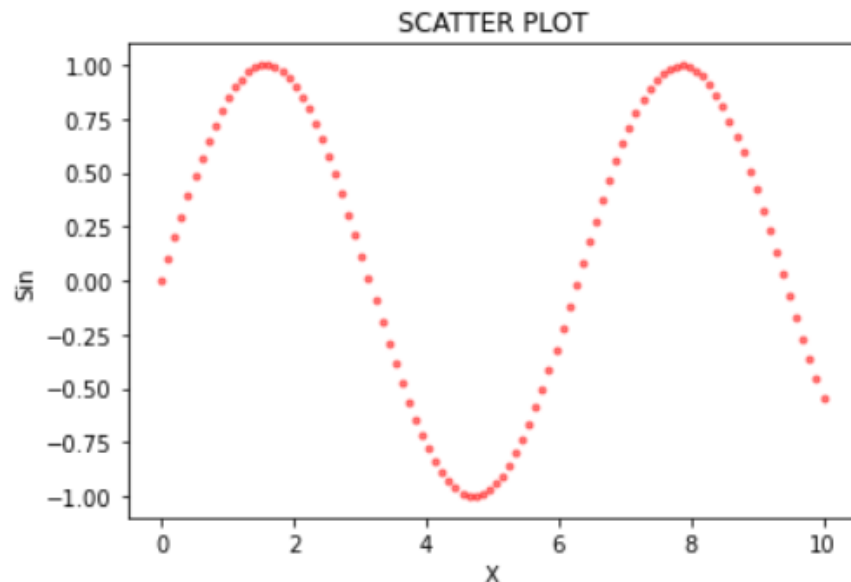
- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **marker**: Símbolo del marcador ("o", ".", "v", "^", "*", etc...)
- **markersize**: Tamaño del marcador
- **linestyle**: none.
- **markerfacecolor**: Color del relleno
- **markeredgecolor**: Color del borde
- **markeredgewidth**: Grosor del borde

Visualización con matplotlib

Visualizaciones básicas

■ Scatter Plots

```
In [14]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.scatter(x,y,c='red',marker='.',alpha=0.5)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



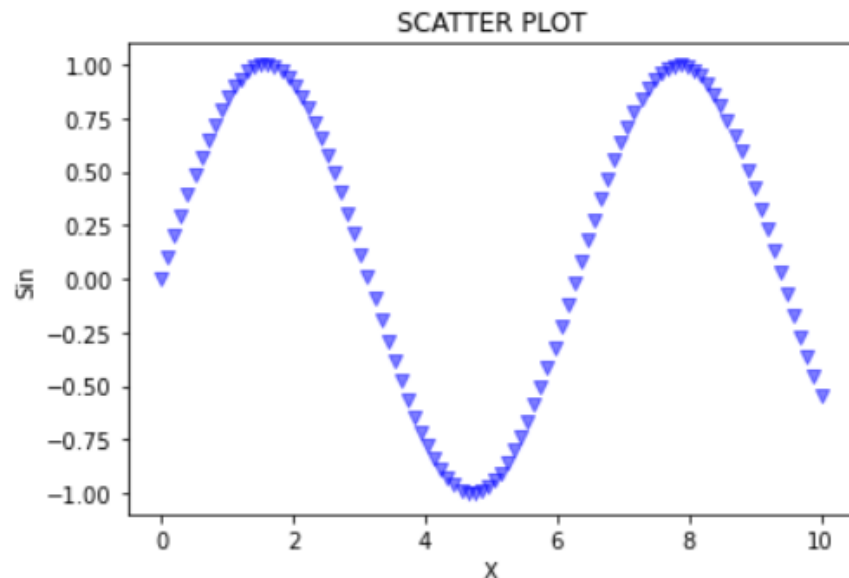
- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **marker**: Símbolo del marcador ("o", ".", "v", "^", "*", etc...)
- **markersize**: Tamaño del marcador
- **linestyle**: none.
- **markerfacecolor**: Color del relleno
- **markeredgecolor**: Color del borde
- **markeredgewidth**: Grosor del borde

Visualización con matplotlib

Visualizaciones básicas

■ Scatter Plots

```
In [15]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.scatter(x,y,c='blue',marker='v',alpha=0.5)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



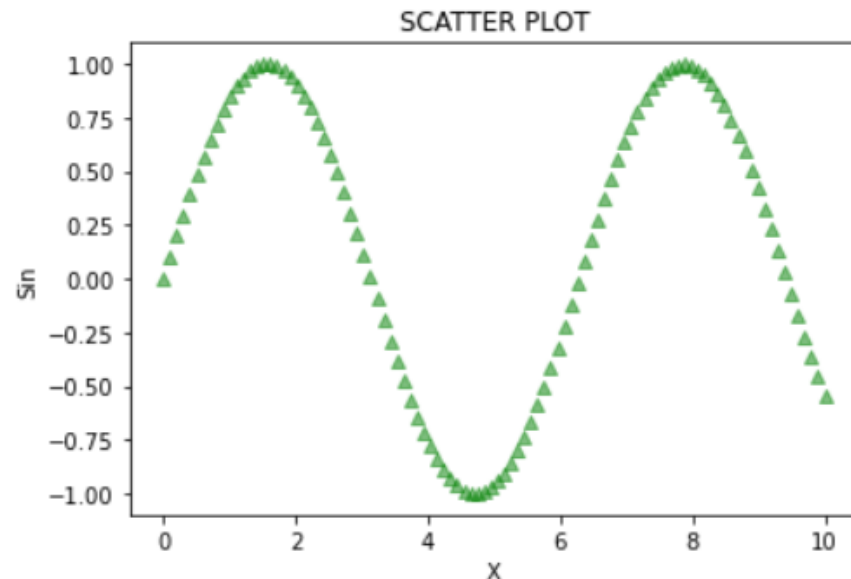
- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **marker**: Símbolo del marcador ("o", ".", "v", "^", "*", etc...)
- **markersize**: Tamaño del marcador
- **linestyle**: none.
- **markerfacecolor**: Color del relleno
- **markeredgecolor**: Color del borde
- **markeredgewidth**: Grosor del borde

Visualización con matplotlib

Visualizaciones básicas

■ Scatter Plots

```
In [16]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.scatter(x,y,c='green',marker='^',alpha=0.5)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



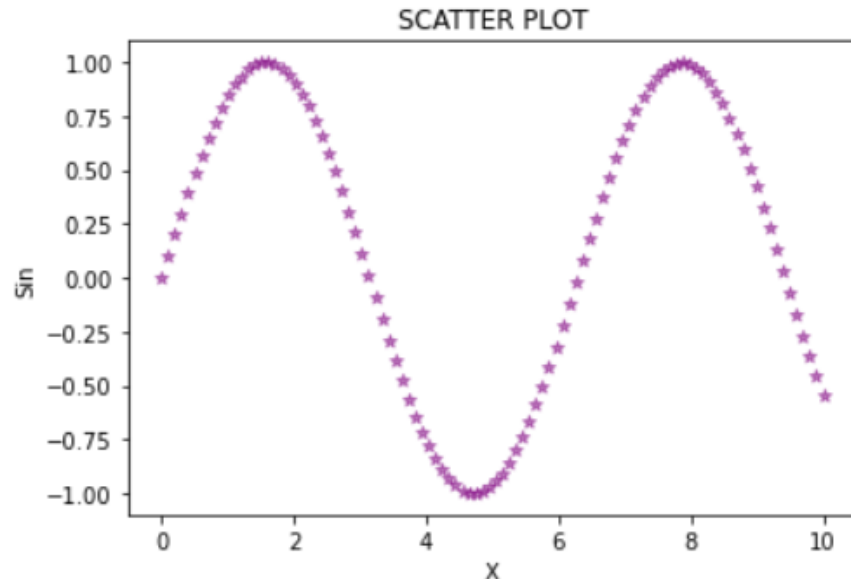
- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **marker**: Símbolo del marcador ("o", ".", "v", "^", "*", etc...)
- **markersize**: Tamaño del marcador
- **linestyle**: none.
- **markerfacecolor**: Color del relleno
- **markeredgecolor**: Color del borde
- **markeredgewidth**: Grosor del borde

Visualización con matplotlib

Visualizaciones básicas

■ Scatter Plots

```
In [18]: x = np.linspace(0,10,100)
y = np.sin(x)
plt.scatter(x,y,c='purple', marker='*', alpha=0.5)
plt.xlabel('X')
plt.ylabel('Sin')
plt.title('SCATTER PLOT')
plt.show()
```



- **x**: Datos a representar en el eje x
- **y**: Datos a representar en el eje y
- **marker**: Símbolo del marcador ("o", ".", "v", "^", "*", etc...)
- **markersize**: Tamaño del marcador
- **linestyle**: none.
- **markerfacecolor**: Color del relleno
- **markeredgecolor**: Color del borde
- **markeredgewidth**: Grosor del borde

Visualización con matplotlib

Visualizaciones básicas

- Argumentos de la función `plot`.

Argumento	Explicación
color	Color
alpha	Transparencia
linewidth	Ancho de línea
linestyle	Estilo de línea
marker	Tipo de marcador
markersize	Tamaño de marcador
markerfacecolor	Color de relleno de marcador
markeredgecolor	Color de borde de marcador
markeredgewidth	Ancho de borde de marcador

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

Visualización con matplotlib

Visualizaciones básicas

- Valores del argumento “estilo de línea” (**linestyle**):

linestyle	Explicación
'none'	Sin línea
'.'	Línea punteada
'--'	Línea discontinua
'-.'	Línea de trazos y puntos
'_'	Línea continua
'steps'	En pasos

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

Visualización con matplotlib

Visualizaciones básicas

- Valores del argumento “estilo de marcador” (**marker**):

marker	Explicación
'.'	Punto
','	Pixel
'o'	Círculo
'^'	Triángulo arriba
'v'	Triángulo abajo
's'	Cuadrado
'*'	Estrella
'+'	Signo más
'x'	Carácter X
'D'	Diamante
'p'	Pentágono

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

Visualización con matplotlib

Visualizaciones con objetos ejes

- Importamos módulos

```
In [1]: import matplotlib.pyplot as plt  
import numpy as np  
%matplotlib inline
```

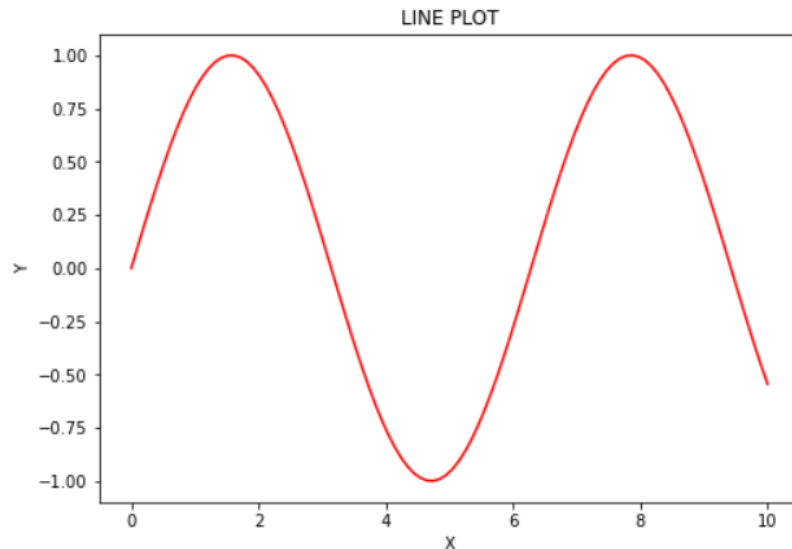
Visualización con matplotlib

Visualizaciones con objetos ejes

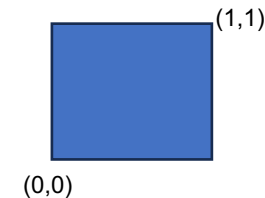
- Creamos los ejes para visualizar figuras

```
In [2]: x = np.linspace(0,10,100)
        y = np.sin(x)
        z = np.cos(x)
```

```
In [3]: fig0=plt.figure()
        axes0 = fig0.add_axes([0,0,1,1]) # Left, bottom, width, height.
        axes0.plot(x,y,color='red',linestyle='-')
        axes0.set_xlabel('X')
        axes0.set_ylabel('Y')
        axes0.set_title('LINE PLOT')
        plt.show()
```



- Se crean tantos **axes** (ejes) como figuras se desean dibujar.
- Los argumentos de **add_axes** son porcentajes de la coordenada inferior izquierda y superior derecha. En el ejemplo de (0,0) a (1,1), es decir, toda la imagen.

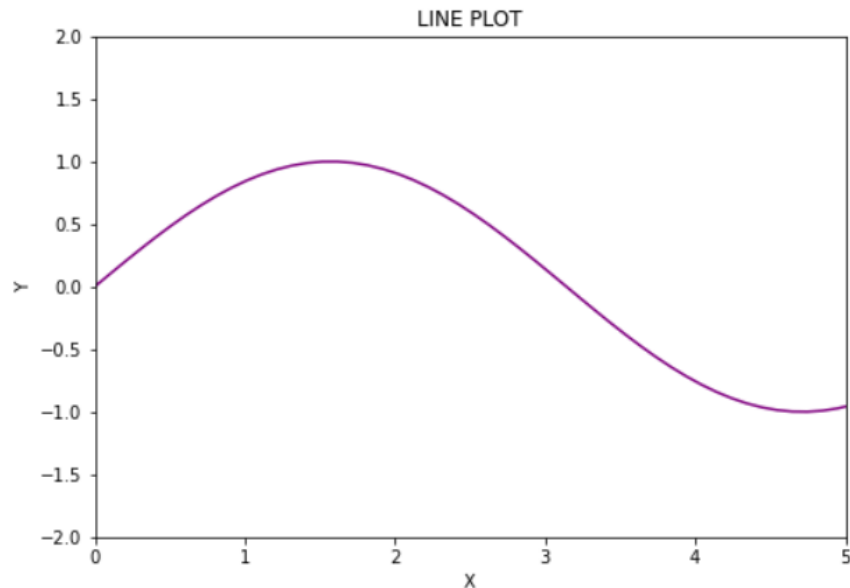


Visualización con matplotlib

Visualizaciones con objetos ejes

- Visualización de una figura

```
In [4]: fig0=plt.figure()
axes0 = fig0.add_axes([0,0,1,1])           # Left, bottom, width, height.
axes0.plot(x,y,color='purple',linestyle='-')
axes0.set_xlabel('X')
axes0.set_ylabel('Y')
axes0.set_title('LINE PLOT')
axes0.set_xlim([0,5])                     # Horizontal axis limits.
axes0.set_ylim([-2,2])                     # Vertical axis limits.
plt.show()
```

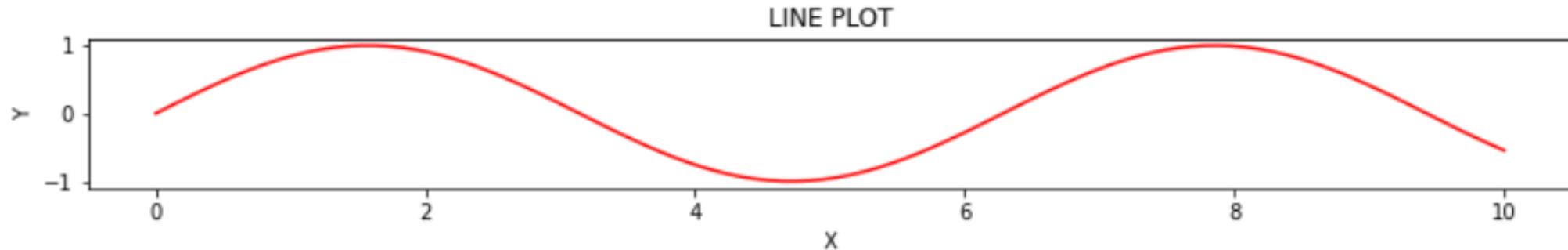


Visualización con matplotlib

Visualizaciones con objetos ejes

- Cambiar el tamaño de una figura

```
In [5]: fig0=plt.figure(figsize=(10,1))           # Width and height specified with figsize.  
axes0 = fig0.add_axes([0,0,1,1])                 # Left, bottom, width, height.  
axes0.plot(x,y,color='red',linestyle='-')  
axes0.set_xlabel('X')  
axes0.set_ylabel('Y')  
axes0.set_title('LINE PLOT')  
plt.show()
```

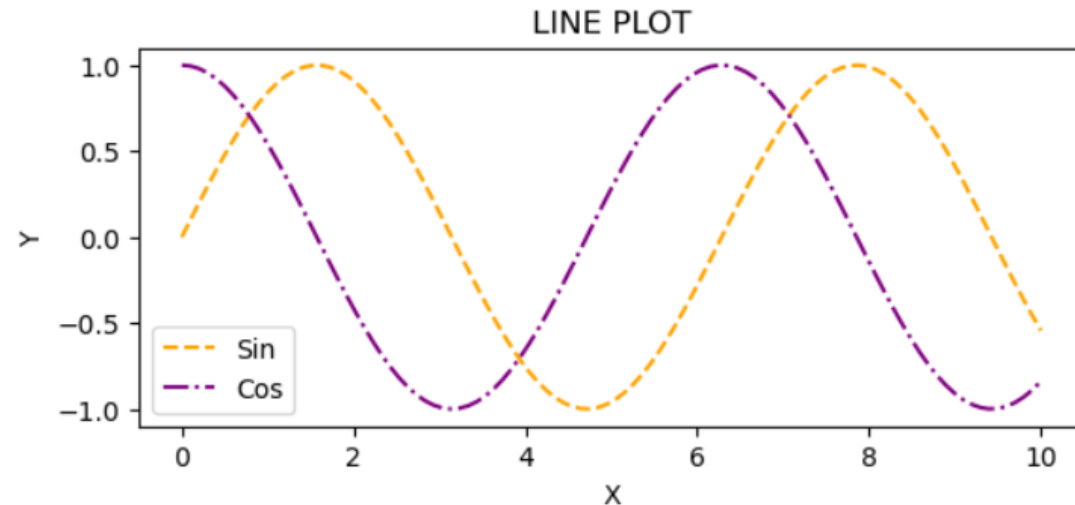


Visualización con matplotlib

Visualizaciones con objetos ejes

- Múltiples figuras en el mismo eje

```
In [6]: fig0=plt.figure(figsize=(5,2), dpi=100)      # Width, height and DPI setting.  
axes0 = fig0.add_axes([0,0,1,1])                  # Left, bottom, width, height.  
axes0.plot(x,y,color='orange',linestyle='--', label='Sin')      # 'label' for the legend.  
axes0.plot(x,z,color='purple',linestyle='-.', label='Cos')      # 'label' for the legend.  
axes0.legend(loc=0)                                     # Legend to be located at the top-right corner.  
axes0.set_xlabel('X')  
axes0.set_ylabel('Y')  
axes0.set_title('LINE PLOT')  
plt.show()
```

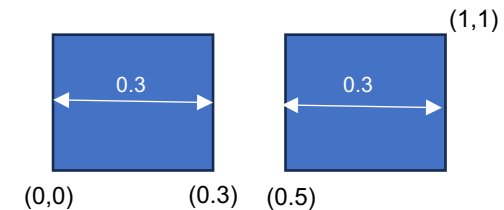
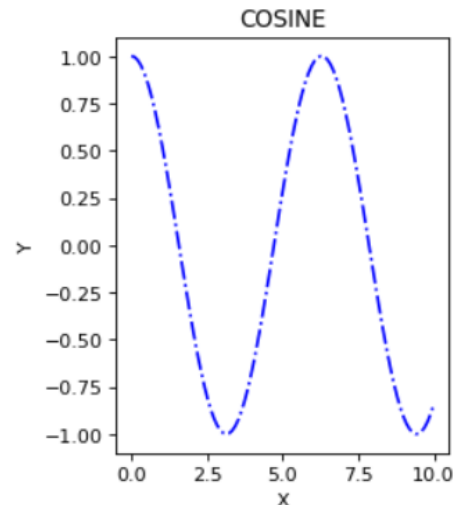
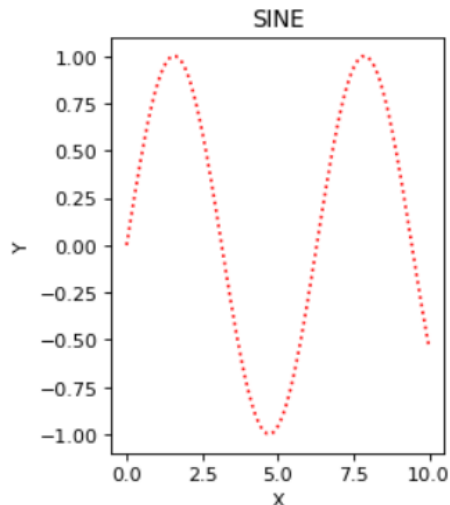


Visualización con matplotlib

Visualizaciones con objetos ejes

- Múltiples figuras en ejes separados

```
In [7]: fig0=plt.figure(figsize=(8,3), dpi=80)      # Width, height and DPI setting.  
axes1 = fig0.add_axes([0,0,0.3,1])                # Left, bottom, width, height.  
axes2 = fig0.add_axes([0.5,0,0.3,1])              # Left, bottom, width, height.  
axes1.plot(x,y,color='red',linestyle=':')  
axes2.plot(x,z,color='blue',linestyle='-.')  
axes1.set_xlabel('X')  
axes1.set_ylabel('Y')  
axes1.set_title('SINE')  
axes2.set_xlabel('X')  
axes2.set_ylabel('Y')  
axes2.set_title('COSINE')  
plt.show()
```

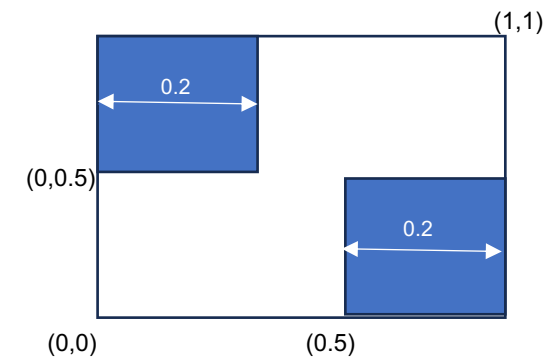
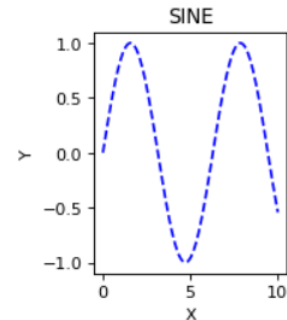
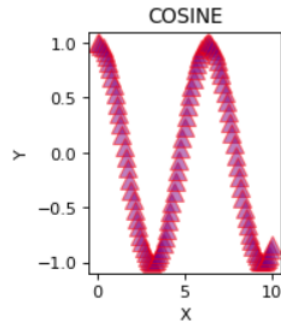


Visualización con matplotlib

Visualizaciones con objetos ejes

- Múltiples figuras en ejes separados

```
In [8]: fig0=plt.figure(figsize=(8,4), dpi=80) # Width, height and DPI setting
axes1 = fig0.add_axes([0,0.5,0.2,0.5]) # Left, bottom, width, height.
axes2 = fig0.add_axes([0.5,0,0.2,0.5]) # Left, bottom, width, height.
axes1.plot(x,z,linestyle='none',marker='^',markerSize=10,markerfacecolor='purple',markeredgecolor='red',alpha=0.5)
axes2.plot(x,y,color='blue',linestyle='--')
axes1.set_xlabel('X')
axes1.set_ylabel('Y')
axes1.set_title('COSINE')
axes2.set_xlabel('X')
axes2.set_ylabel('Y')
axes2.set_title('SINE')
plt.show()
```



Visualización con matplotlib

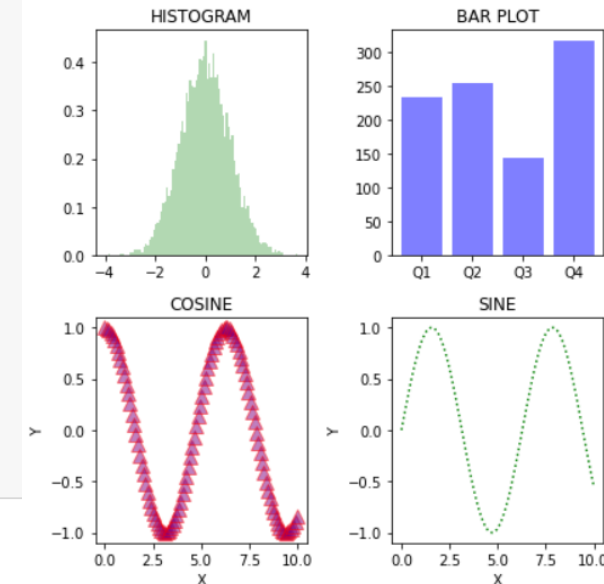
Visualizaciones con objetos ejes

- Múltiples figuras en array de ejes de **ncols** y **nrows**

```
In [9]: fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(6,6))
# (0,0)
c = np.random.randn(10000)
axes[0,0].hist(c,bins=100,color='green',density=True,alpha=0.3)
axes[0,0].set_title('HISTOGRAM')
# (1,0)
axes[1,0].plot(x,z,linestyle='none',marker='^',markersize=10,markerfacecolor='purple',markeredgecolor='red',alpha=0.5)
axes[1,0].set_xlabel('X')
axes[1,0].set_ylabel('Y')
axes[1,0].set_title('COSINE')
# (0,1)
a = np.array(['Q1', 'Q2', 'Q3', 'Q4'])
b = np.array([ 234.0, 254.7, 144.6, 317.6])
axes[0,1].bar(a,b,color='blue',alpha=0.5)
axes[0,1].set_title('BAR PLOT')
# (1,1)
axes[1,1].plot(x,y,color='green',linestyle=':')
axes[1,1].set_xlabel('X')
axes[1,1].set_ylabel('Y')
axes[1,1].set_title('SINE')

plt.tight_layout()
plt.show()
```

Avoid overlapping.



Visualización con matplotlib

Visualizaciones con pandas

- Partimos de un DataFrame

```
In [1]: import pandas as pd  
import numpy as np  
import os  
import matplotlib.pyplot as plt  
%matplotlib inline
```

```
In [2]: df = pd.read_csv('data_iris.csv', header='infer')
```

```
In [3]: df.shape
```

```
Out[3]: (150, 5)
```

```
In [4]: df.head(5)
```

```
Out[4]:
```

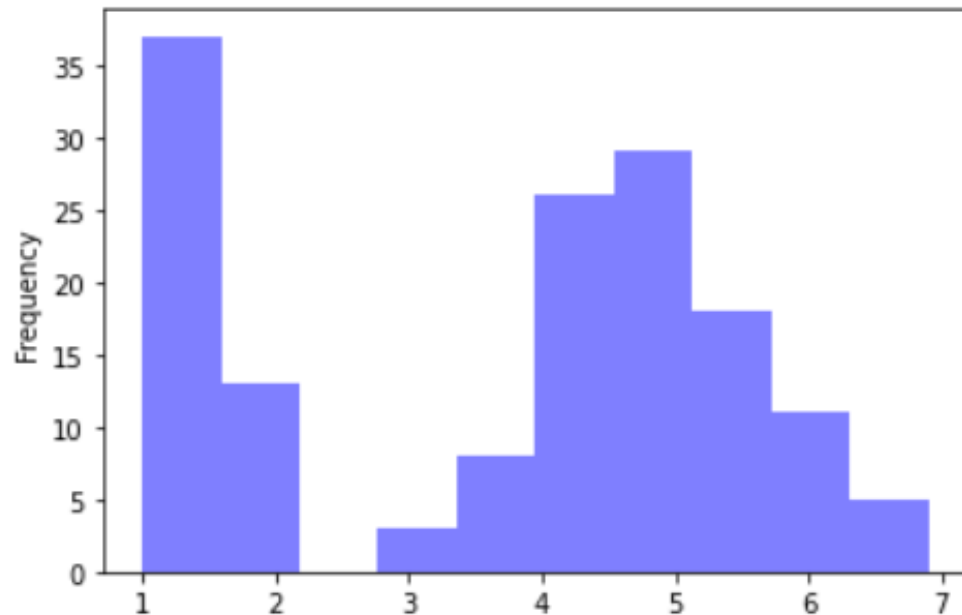
	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Visualización con matplotlib

Visualizaciones con pandas

- Histograma de un DataFrame

```
In [5]: df.loc[:, 'Petal.Length'].plot.hist(color='blue', alpha=0.5)  
plt.show()
```



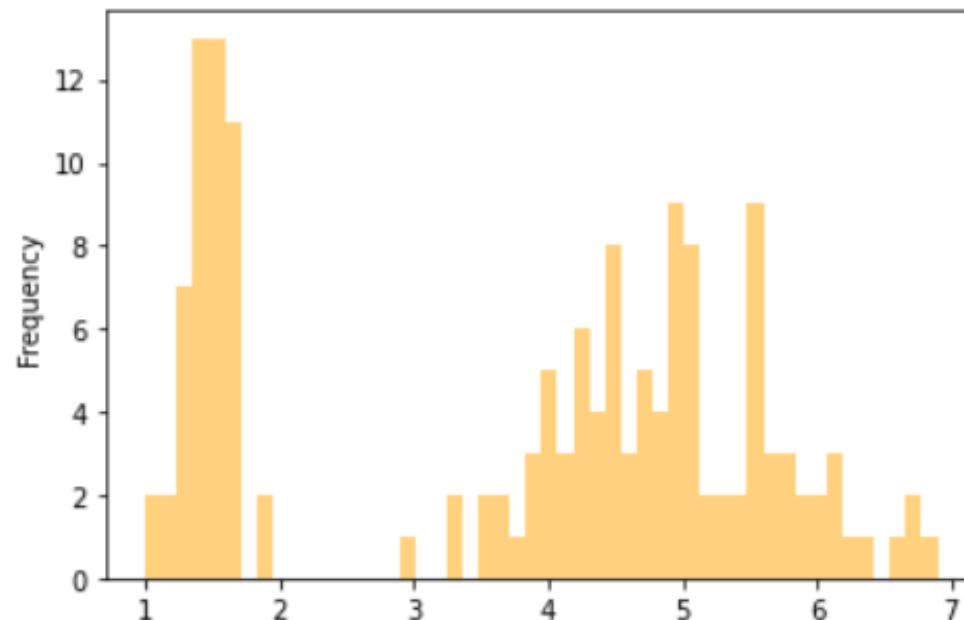
- El módulo **plot** de un DataFrame o Series sigue un API muy similar a **matplotlib**.
- Toma los datos directamente del **DataFrame** o **Series**.

Visualización con matplotlib

Visualizaciones con pandas

- Histograma de un DataFrame

```
In [6]: df.loc[:, 'Petal.Length'].plot.hist(bins=50, color='orange', alpha=0.5)  
plt.show()
```



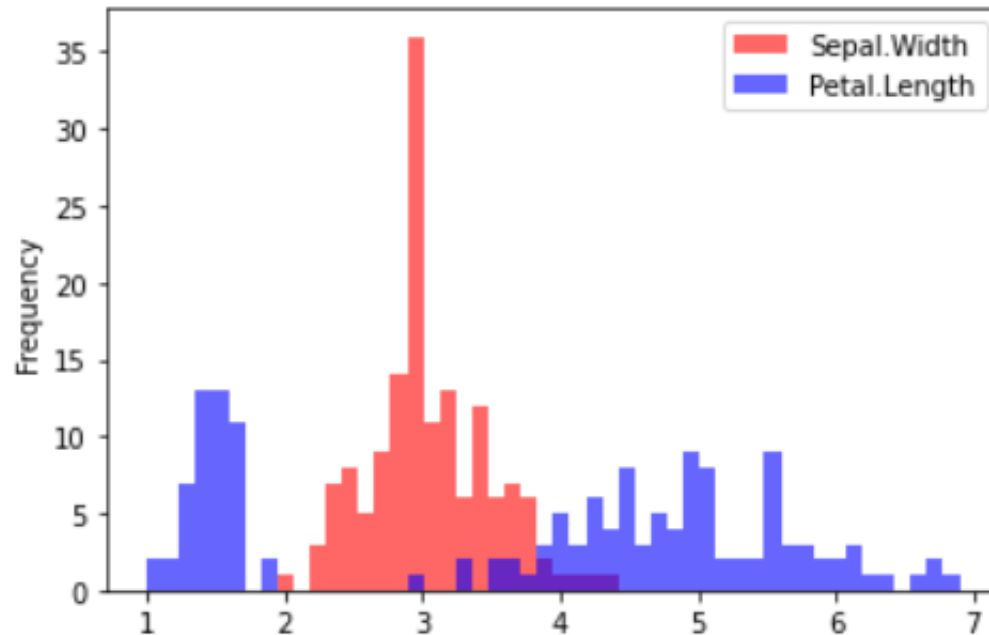
- El módulo **plot** de un DataFrame o Series sigue un API muy similar a **matplotlib**.
- Toma los datos directamente del **DataFrame** o **Series**.

Visualización con matplotlib

Visualizaciones con pandas

■ Histograma de un DataFrame

```
In [7]: df.loc[:,['Sepal.Width','Petal.Length']].plot.hist(bins=50,color=['red','blue'],alpha=0.6)
plt.show()
```



- El módulo **plot** de un DataFrame o Series sigue un API muy similar a **matplotlib**.
- Toma los datos directamente del **DataFrame** o **Series**.
- Si seleccionamos más de una columna dibuja **múltiples gráficas** en el mismo eje.

Visualización con matplotlib

Visualizaciones con pandas

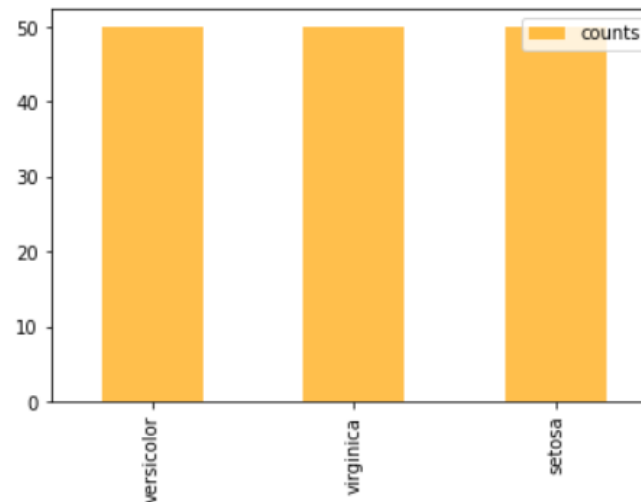
■ Gráfico de Barras de un DataFrame

```
In [8]: frequencies = df.Species.value_counts()
my_counts = list(frequencies.values)
my_labels = list(frequencies.index)
df2 = pd.DataFrame( {'counts':my_counts}, index = my_labels)
df2
```

Out[8]:

	counts
versicolor	50
virginica	50
setosa	50

```
In [9]: df2.plot.bar(color='orange', alpha=0.7)
plt.show()
```

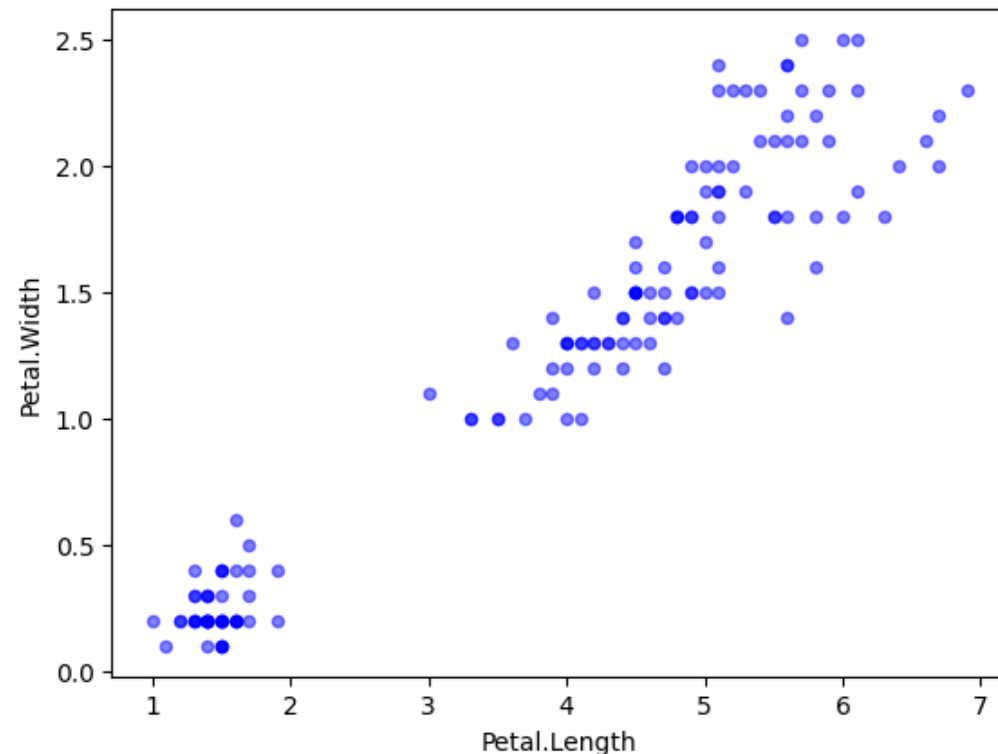


Visualización con matplotlib

Visualizaciones con pandas

- Scatter Plot de un DataFrame

```
In [10]: df.plot.scatter(x='Petal.Length', y='Petal.Width', color='blue', alpha=0.5, marker='o')  
plt.show()
```

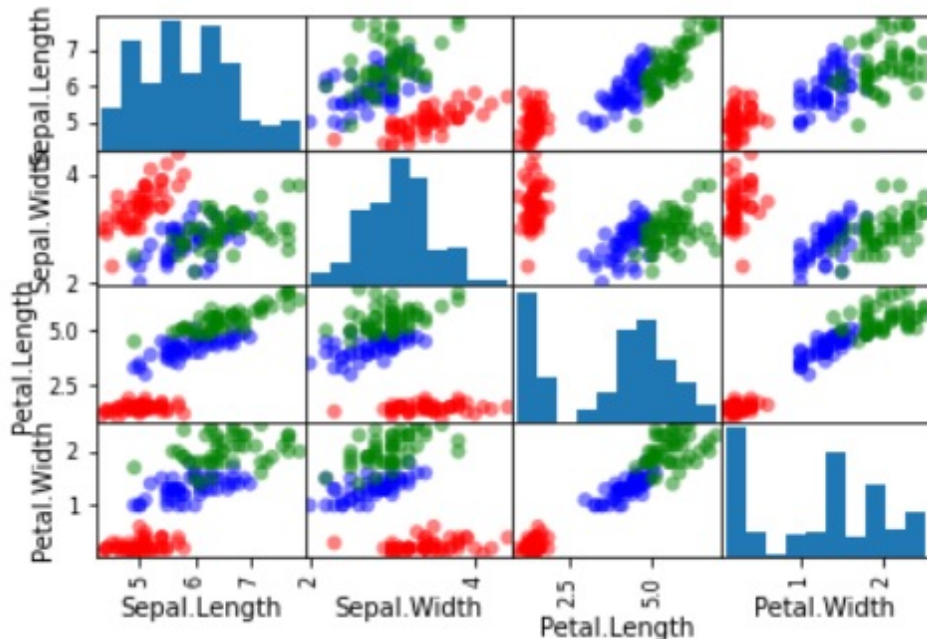


Visualización con matplotlib

Visualizaciones con pandas

- Array de Scatter Plots de un DataFrame

```
In [11]: my_cols_dict = {'setosa':'red', 'virginica':'green', 'versicolor':'blue'}  
my_cols = df['Species'].apply(lambda x: my_cols_dict[x])  
pd.plotting.scatter_matrix(df, c=my_cols, marker='o', alpha=0.5)  
plt.show()
```



Visualización con seaborn

Instalar seaborn

- Importar módulos

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import os
import warnings
%matplotlib inline
warnings.filterwarnings('ignore')           # Turn off the warnings.
```

Visualización con seaborn

Seaborn

- Importar dataset de ejemplo

```
In [2]: dat = sns.load_dataset('mpg')
```

```
In [3]: dat.shape
```

```
Out[3]: (398, 9)
```

```
In [4]: dat.head(5)
```

```
Out[4]:
```

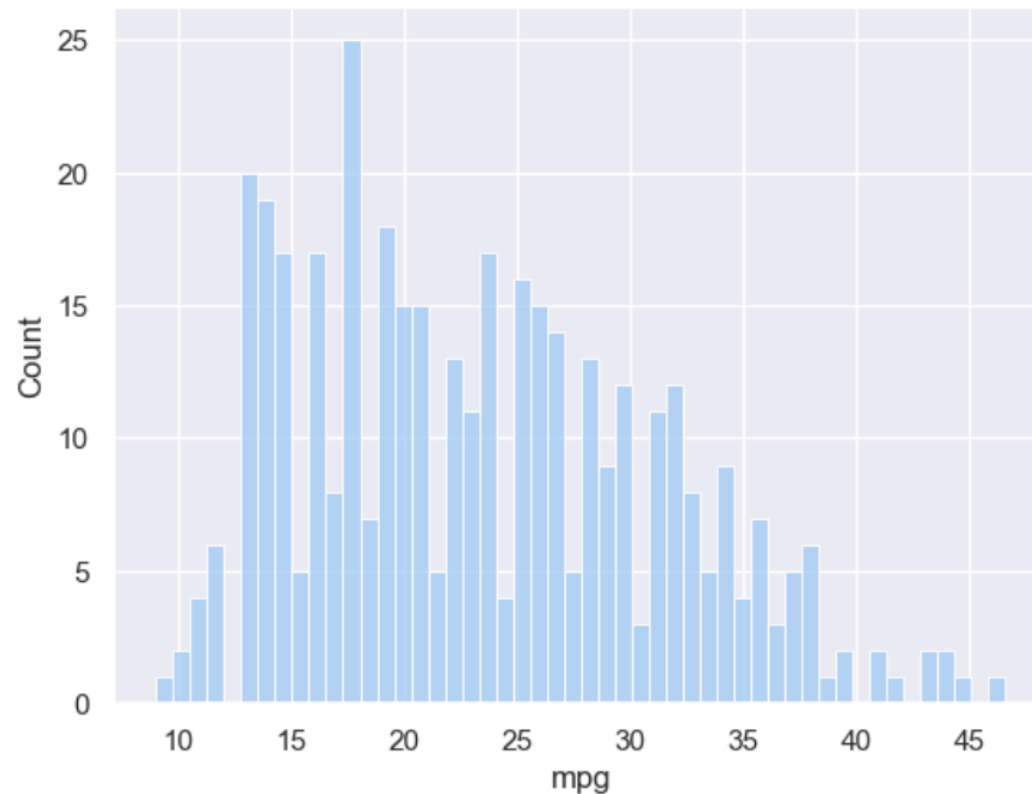
	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130.0	3504	12.0	70	usa	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693	11.5	70	usa	buick skylark 320
2	18.0	8	318.0	150.0	3436	11.0	70	usa	plymouth satellite
3	16.0	8	304.0	150.0	3433	12.0	70	usa	amc rebel sst
4	17.0	8	302.0	140.0	3449	10.5	70	usa	ford torino

Visualización con seaborn

Seaborn

■ Histograma con seaborn

```
[19]: sns.histplot(dat.mpg, bins=50)  
plt.show()
```



- **bins:** Cantidad de grupos del histograma

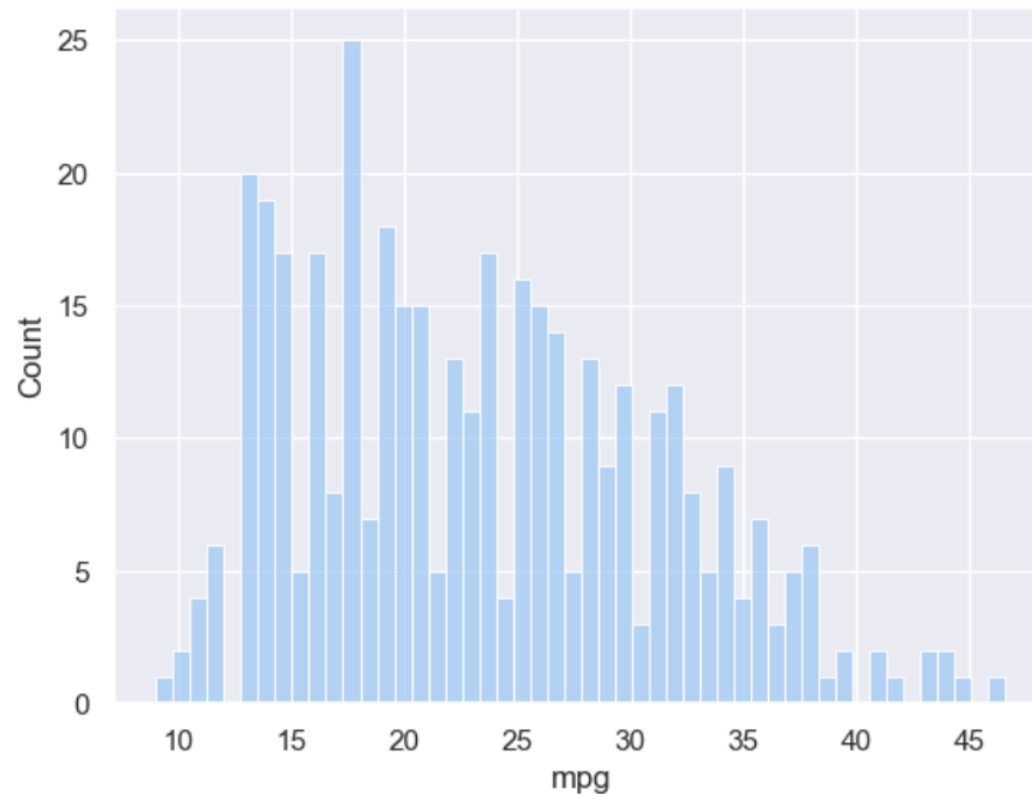
```
seaborn.histplot(data=None, *, x=None, y=None, hue=None, weights=None,  
stat='count', bins='auto', binwidth=None, binrange=None, discrete=None,  
cumulative=False, common_bins=True, common_norm=True, multiple='layer',  
element='bars', fill=True, shrink=1, kde=False, kde_kws=None, line_kws=None,  
thresh=0, pthresh=None, pmax=None, cbar=False, cbar_ax=None, cbar_kws=None,  
palette=None, hue_order=None, hue_norm=None, color=None, log_scale=None,  
legend=True, ax=None, **kwargs)
```

Visualización con seaborn

Seaborn

■ Histograma con seaborn

```
[19]: sns.histplot(dat.mpg, bins=50)  
plt.show()
```



- **bins:** Cantidad de grupos del histograma

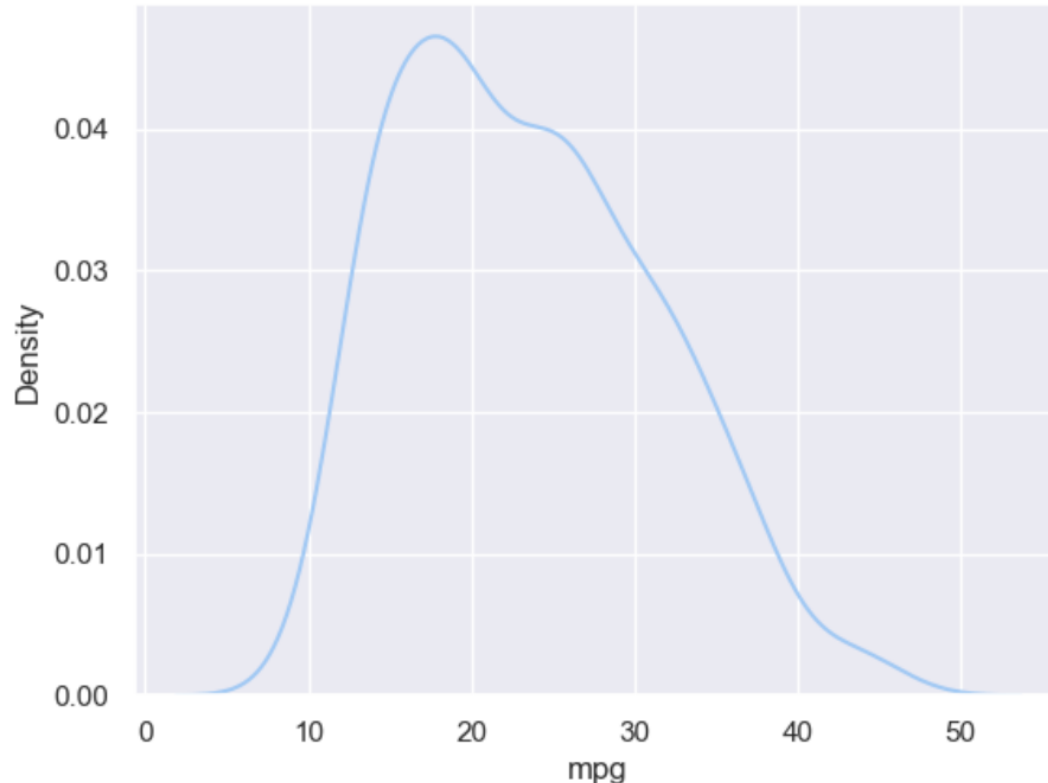
```
seaborn.histplot(data=None, *, x=None, y=None, hue=None, weights=None,  
stat='count', bins='auto', binwidth=None, binrange=None, discrete=None,  
cumulative=False, common_bins=True, common_norm=True, multiple='layer',  
element='bars', fill=True, shrink=1, kde=False, kde_kws=None, line_kws=None,  
thresh=0, pthresh=None, pmax=None, cbar=False, cbar_ax=None, cbar_kws=None,  
palette=None, hue_order=None, hue_norm=None, color=None, log_scale=None,  
legend=True, ax=None, **kwargs)
```

Visualización con seaborn

Seaborn

- Kernel Density Estimate (KDE) con seaborn

```
[21]: sns.kdeplot(dat.mpg)
plt.show()
```



- Un gráfico de Kernel Density Estimate (KDE) es un método para visualizar la **distribución de observaciones** en un conjunto de datos, análogo a un histograma.
- El KDE representa los datos mediante una **curva continua de densidad de probabilidad**, en una o varias dimensiones.

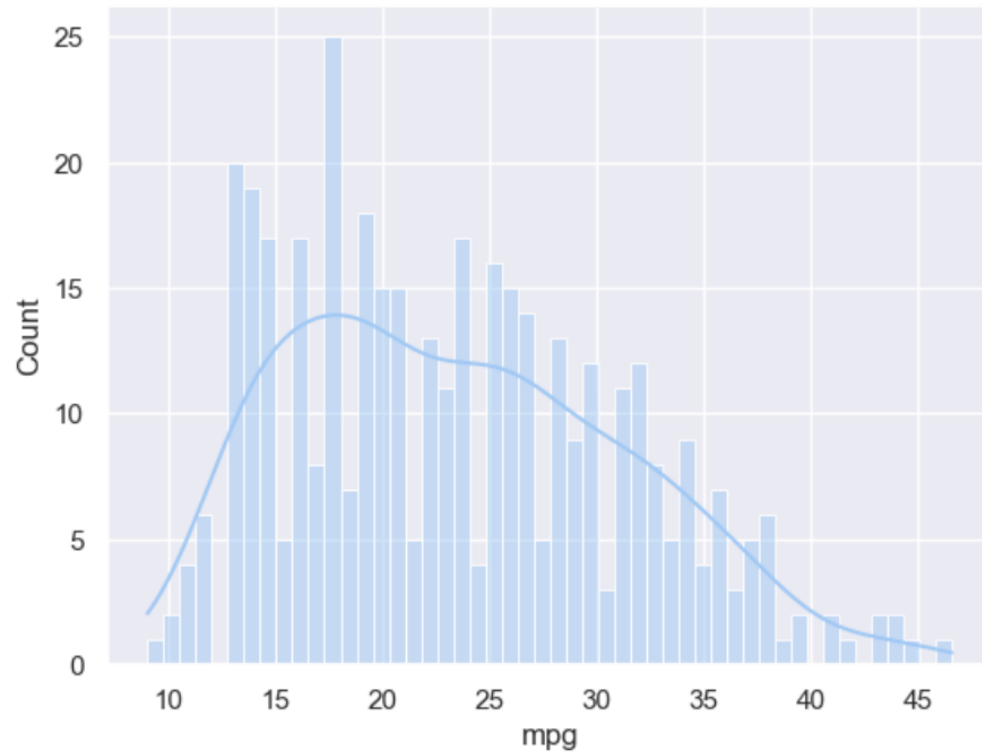
```
seaborn.kdeplot(data=None, *, x=None, y=None, hue=None, weights=None,
palette=None, hue_order=None, hue_norm=None, color=None, fill=None,
multiple='layer', common_norm=True, common_grid=False, cumulative=False,
bw_method='scott', bw_adjust=1, warn_singular=True, log_scale=None, levels=10,
thresh=0.05, gridsize=200, cut=3, clip=None, legend=True, cbar=False, cbar_ax=None,
cbar_kws=None, ax=None, **kwargs)
```

Visualización con seaborn

Seaborn

- Histograma con KDE en seaborn

```
[23]: sns.histplot(dat.mpg, kde=True, bins=50)  
plt.show()
```



Es posible sobreimpresionar KDE sobre un histograma.

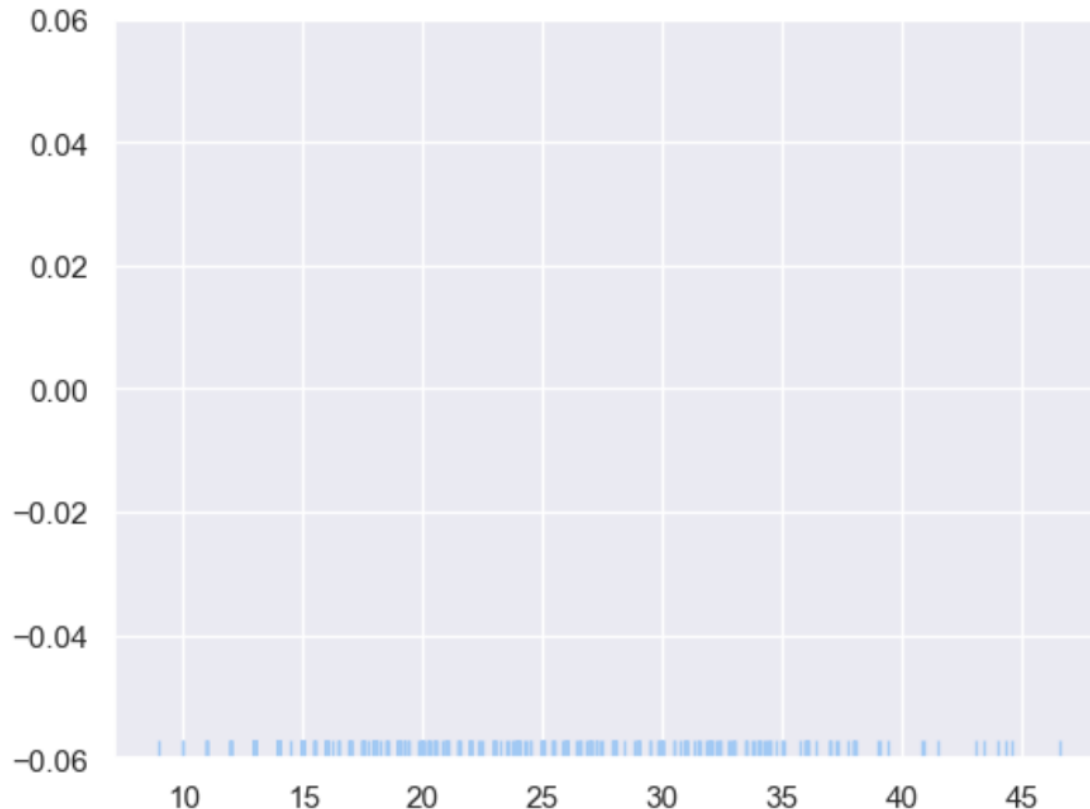
```
seaborn.histplot(data=None, *, x=None, y=None, hue=None, weights=None,  
stat='count', bins='auto', binwidth=None, binrange=None, discrete=None,  
cumulative=False, common_bins=True, common_norm=True, multiple='layer',  
element='bars', fill=True, shrink=1, kde=False, kde_kws=None, line_kws=None,  
thresh=0, pthresh=None, pmax=None, cbar=False, cbar_ax=None, cbar_kws=None,  
palette=None, hue_order=None, hue_norm=None, color=None, log_scale=None,  
legend=True, ax=None, **kwargs)
```

Visualización con seaborn

Seaborn

■ Rugplot en seaborn

```
[25]: sns.rugplot(dat.mpg)
plt.show()
```



- Muestra observaciones **individuales** de un conjunto de datos como **pequeñas marcas (ticks)** a lo largo de un eje.
- Cada dato se representa como una **rayita vertical** u horizontal.
- Normalmente se coloca en el eje X, y se puede **combinar** con histogramas o KDE para mostrar dónde se concentran los datos.
- Sirve como una **representación densa y compacta** de la distribución de los valores.

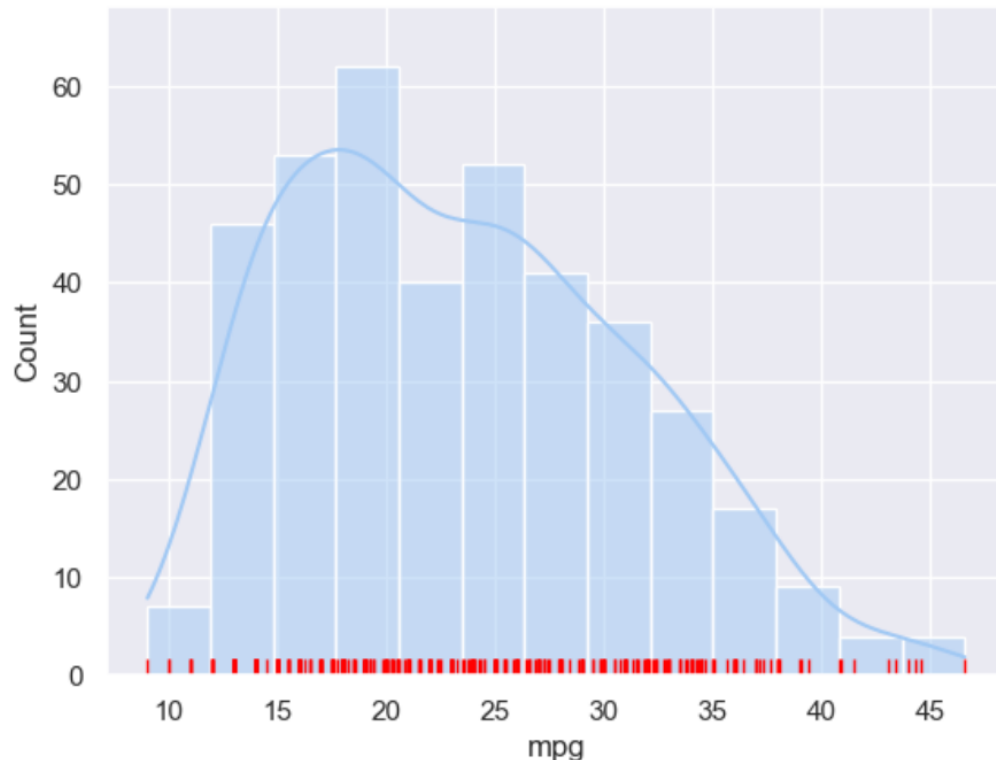
```
seaborn.rugplot(data=None, *, x=None, y=None, hue=None, height=0.025,
expand_margins=True, palette=None, hue_order=None, hue_norm=None, legend=True,
ax=None, **kwargs)
```

Visualización con seaborn

Seaborn

■ Rugplot en seaborn

```
[29]: sns.histplot(dat.mpg, kde=True)  
sns.rugplot(dat.mpg, color="red")  
plt.show()
```



- Muestra observaciones **individuales** de un conjunto de datos como **pequeñas marcas (ticks)** a lo largo de un eje.
- Cada dato se representa como una **rayita vertical** u horizontal.
- Normalmente se coloca en el eje X, y se puede **combinar** con histogramas o KDE para mostrar dónde se concentran los datos.
- Sirve como una **representación densa y compacta** de la distribución de los valores.

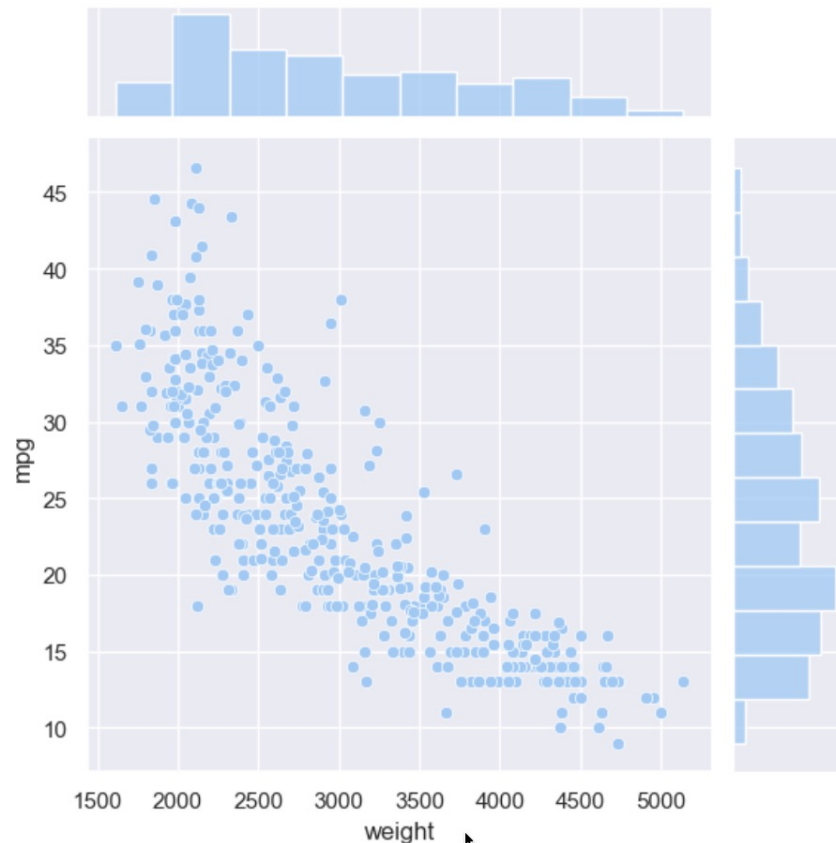
```
seaborn.rugplot(data=None, *, x=None, y=None, hue=None, height=0.025,  
expand_margins=True, palette=None, hue_order=None, hue_norm=None, legend=True,  
ax=None, **kwargs)
```

Visualización con seaborn

Seaborn

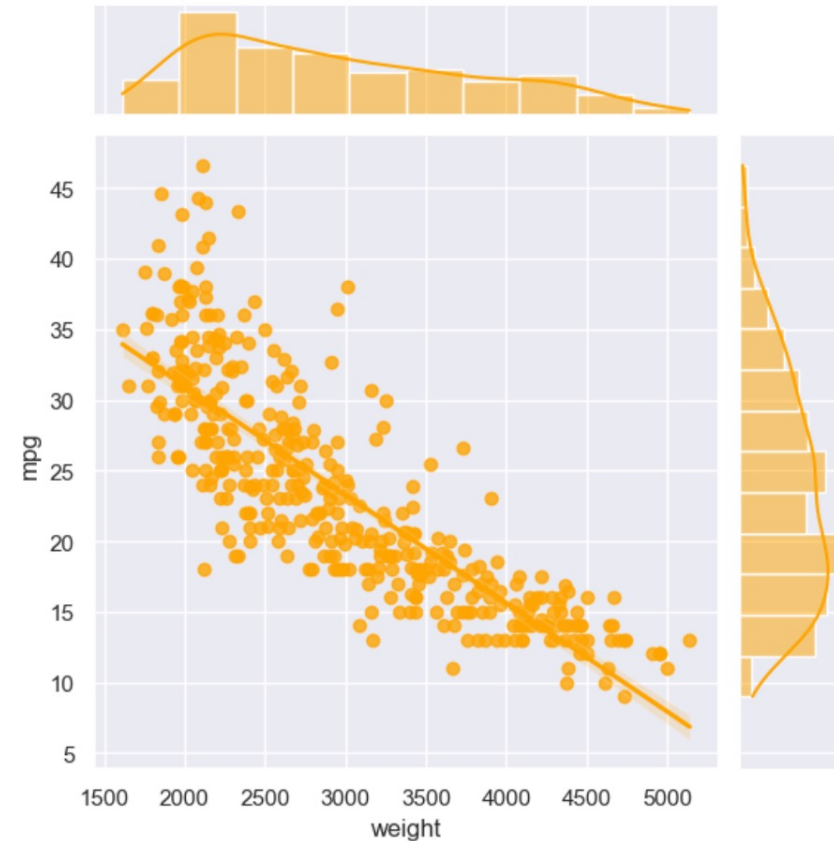
■ Jointplot en seaborn

```
[31]: sns.jointplot(x="weight", y="mpg", data=dat, kind="scatter")  
plt.show()
```



- Representa dos variables mostrando tanto su **relación conjunta** (bivariada) como sus **distribuciones individuales** (univariadas).

```
[32]: sns.jointplot(x="weight", y="mpg", data=dat, kind="reg", color="orange")  
plt.show()
```

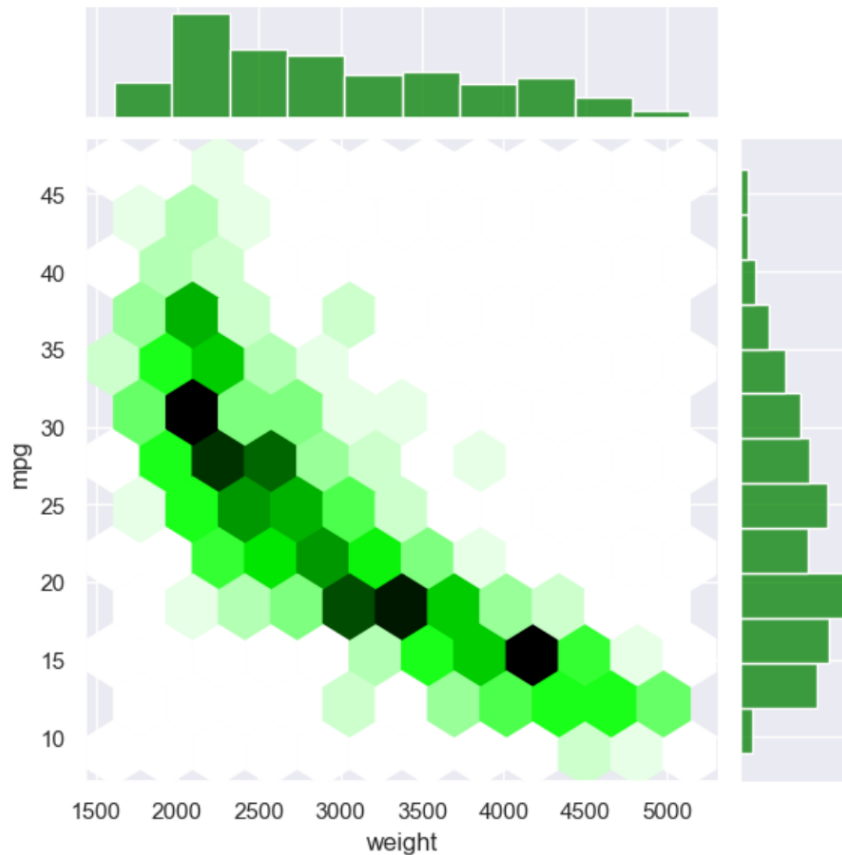


Visualización con seaborn

Seaborn

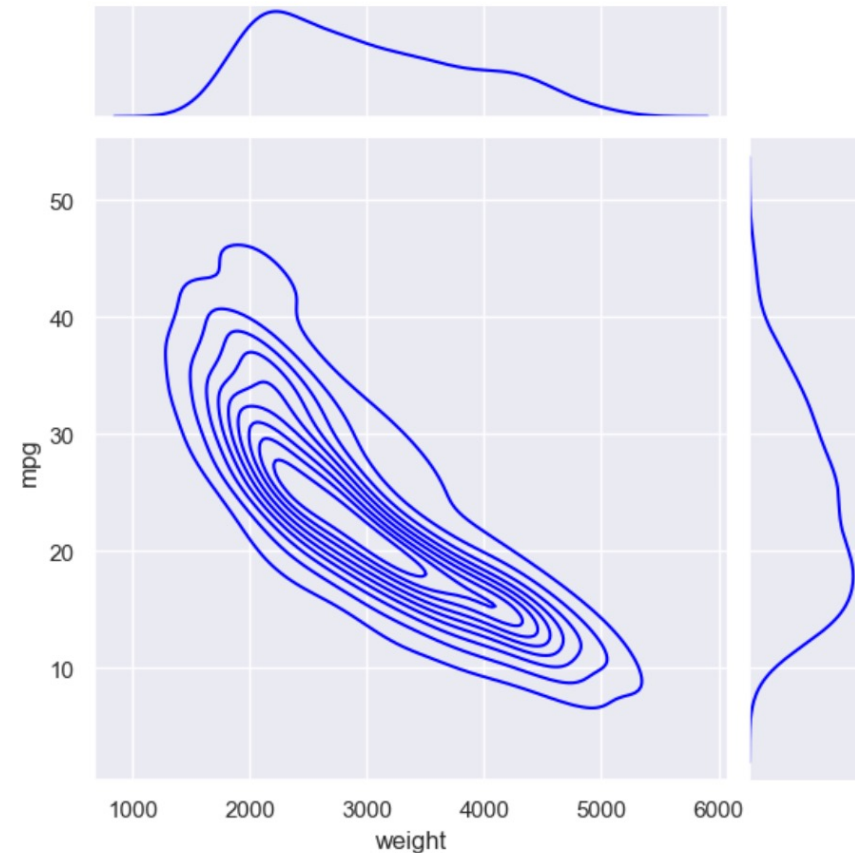
■ Jointplot en seaborn

```
[41]: sns.jointplot(x="weight", y="mpg", data=dat, color="green", kind="hex")  
plt.show()
```



- Representa dos variables mostrando tanto su **relación conjunta** (bivariada) como sus **distribuciones individuales** (univariadas).

```
[42]: sns.jointplot(x="weight", y="mpg", data=dat, color="blue", kind="kde")  
plt.show()
```

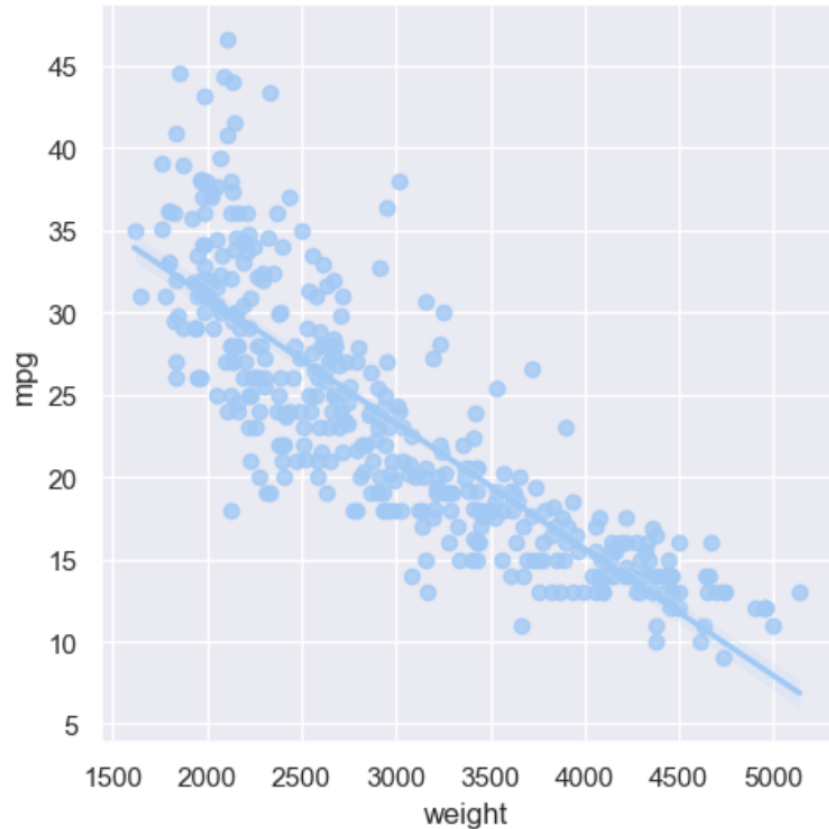


Visualización con seaborn

Seaborn

■ Implot en seaborn

```
[33]: sns.lmplot(x="weight", y="mpg", data=dat)  
plt.show()
```



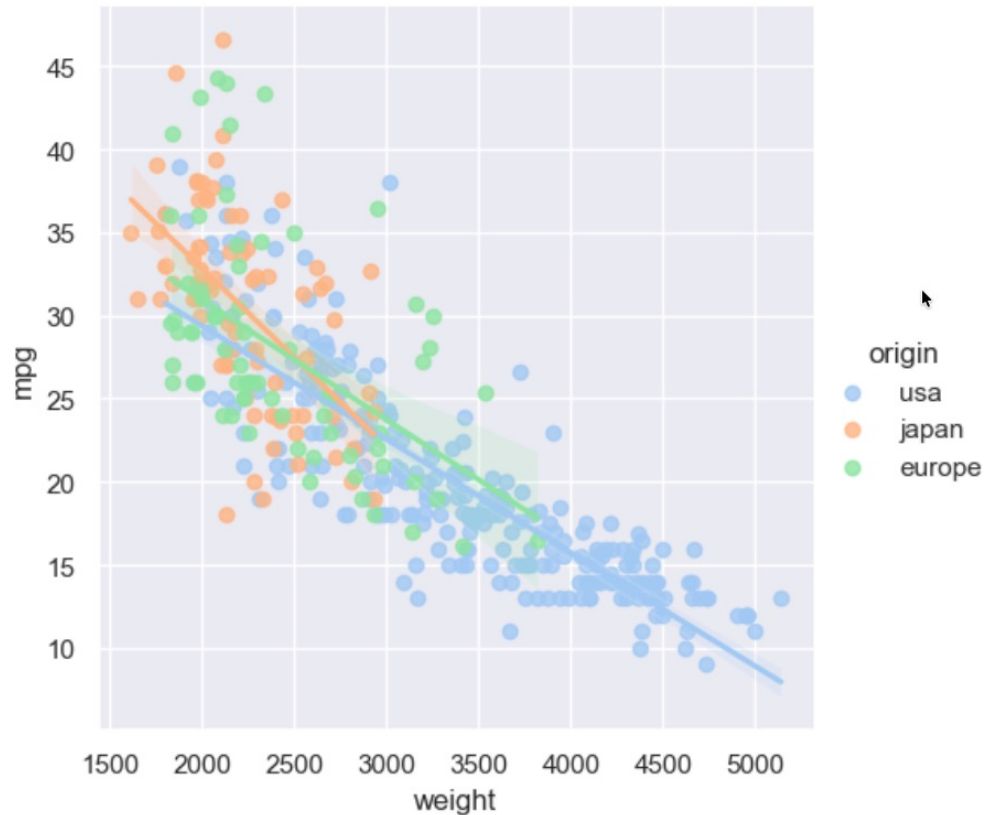
- Representa los datos y el modelo de regresión.

Visualización con seaborn

Seaborn

■ Implot en seaborn

```
[36]: sns.lmplot(x="weight", y="mpg", data=dat, hue="origin")  
plt.show()
```



- Representa los datos y el modelo de regresión.
- **hue**: Separa una variable por colores

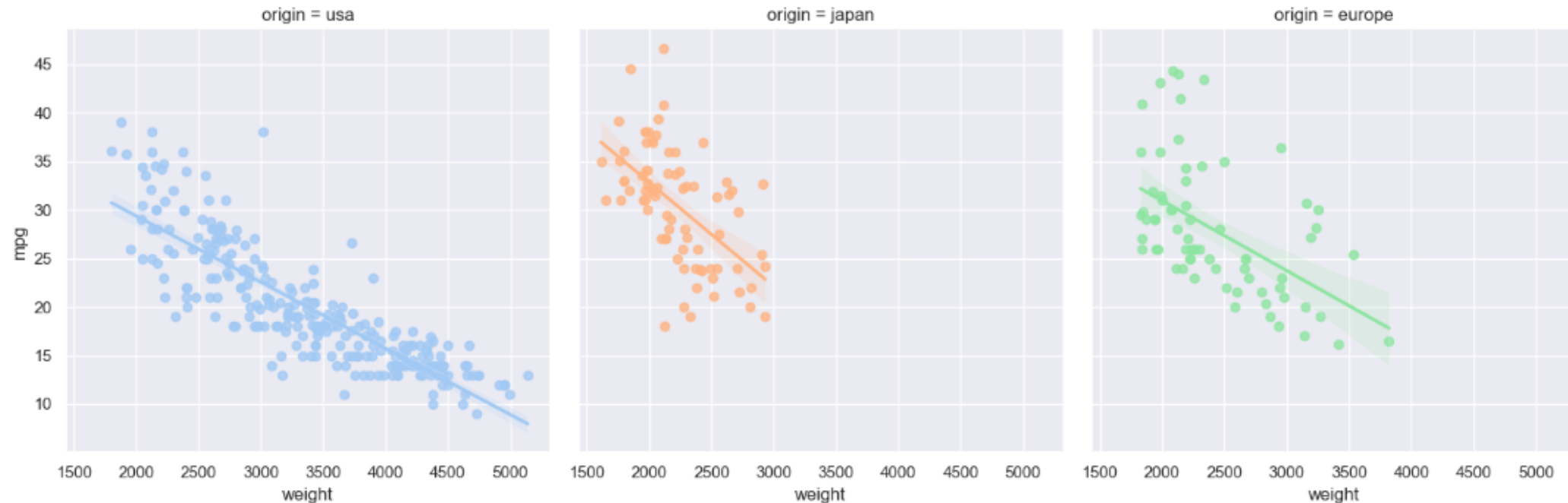
Visualización con seaborn

Seaborn

■ Implot en seaborn

```
[37]: sns.lmplot(x="weight", y="mpg", data=dat, hue="origin", col="origin")  
plt.show()
```

- Representa los datos y el modelo de regresión.
- **hue**: Separa una variable por colores
- **col**: Separa una variable en columnas.

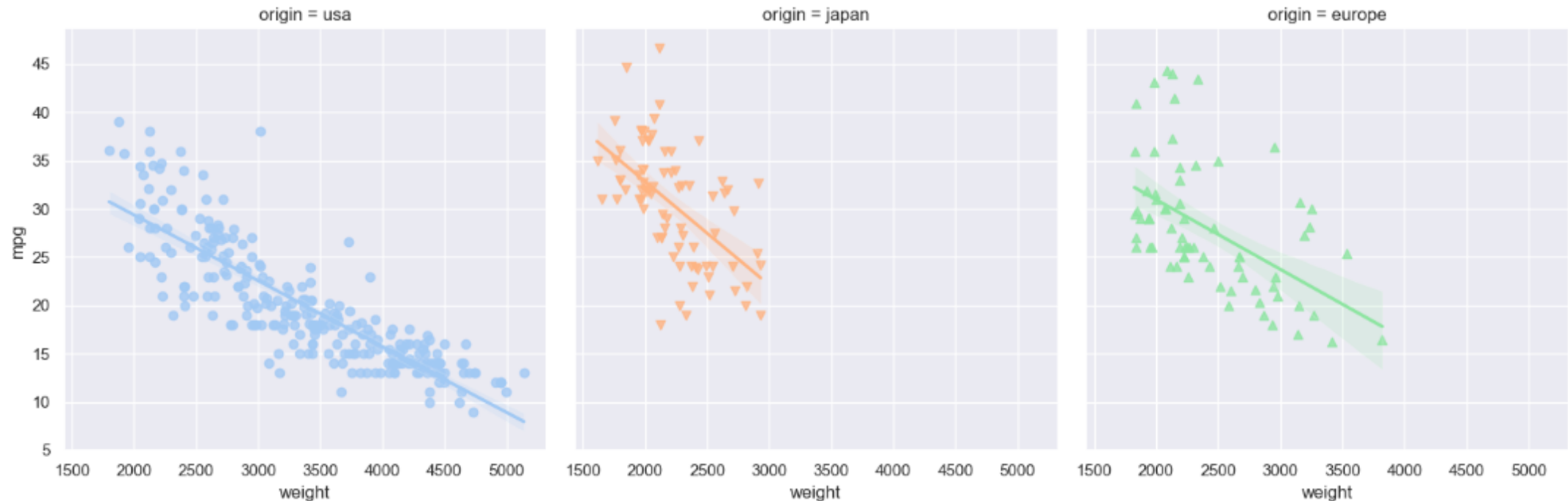


Visualización con seaborn

Seaborn

■ Implot en seaborn

```
[38]: sns.lmplot(x="weight", y="mpg", data=dat, hue="origin", col="origin",  
               markers=["o", "v", "^"])  
plt.show()
```



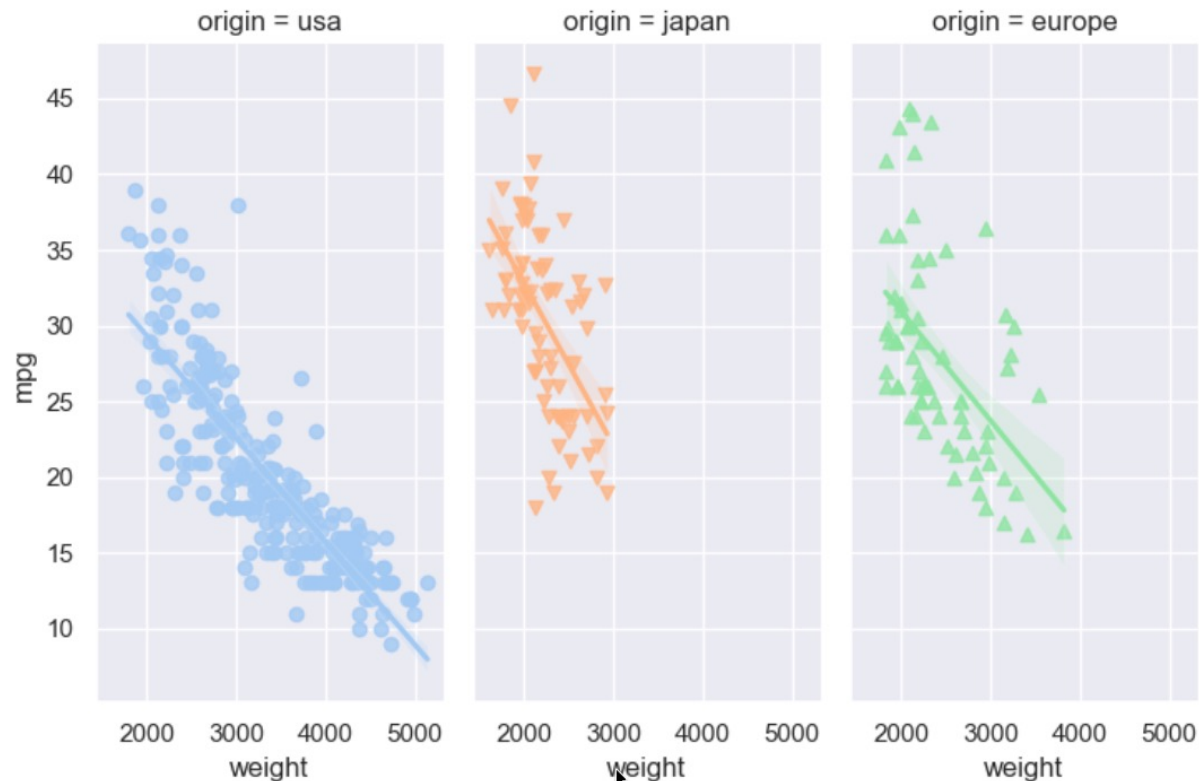
- Representa los datos y el modelo de regresión.
- **hue:** Separa una variable por colores
- **col:** Separa una variable en columnas.

Visualización con seaborn

Seaborn

■ Implot en seaborn

```
[39]: sns.lmplot(x="weight", y="mpg", data=dat, hue="origin", col="origin",  
               markers=["o", "v", "^"], aspect=0.5)  
plt.show()
```



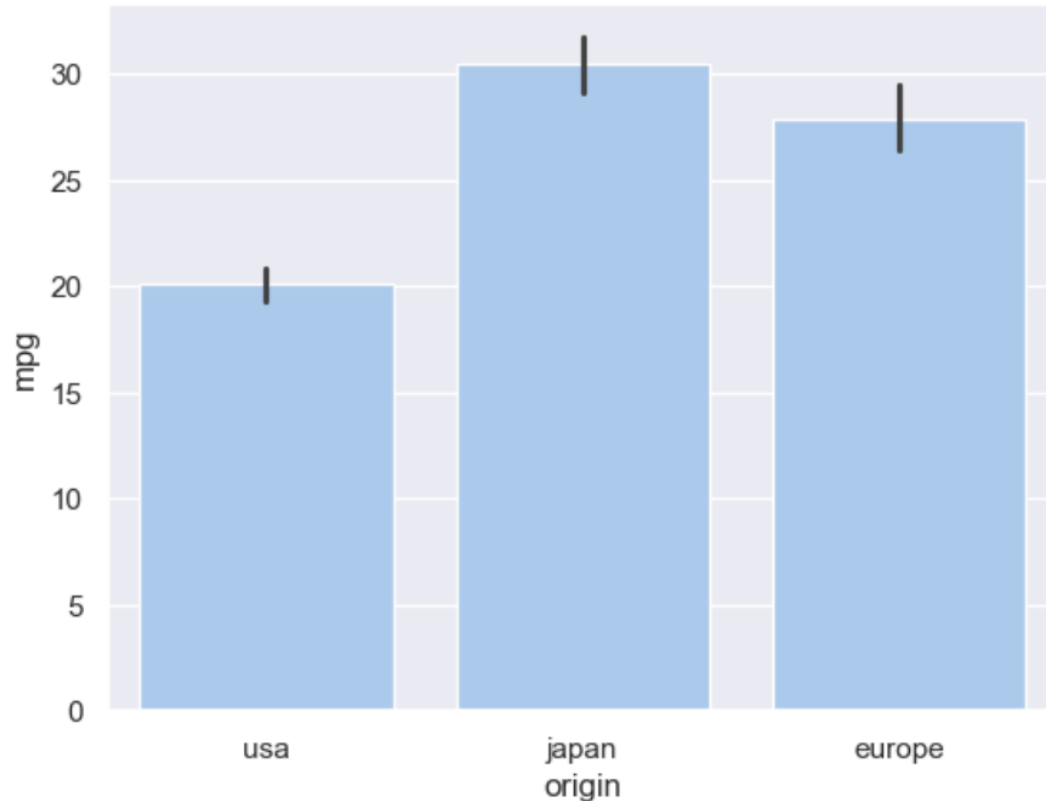
- Representa los datos y el modelo de regresión.
- **hue:** Separa una variable por colores
- **col:** Separa una variable en columnas.

Visualización con seaborn

Seaborn

■ barplot en seaborn

```
[43]: sns.barplot(x="origin", y="mpg", data=dat)  
plt.show()
```



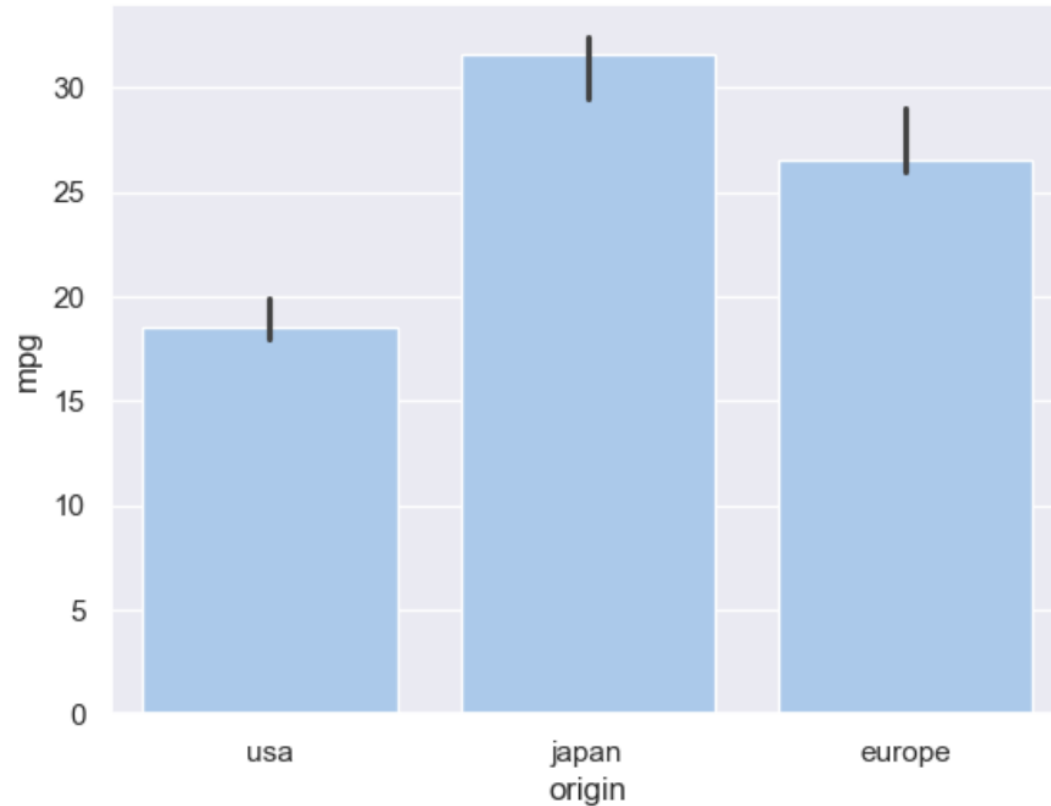
- Una variable **categorica**
- Una variable **numérica**
- **Agregadas** por la media

Visualización con seaborn

Seaborn

■ barplot en seaborn

```
[45]: sns.barplot(x="origin", y="mpg", data=dat, estimator=np.median)  
plt.show()
```



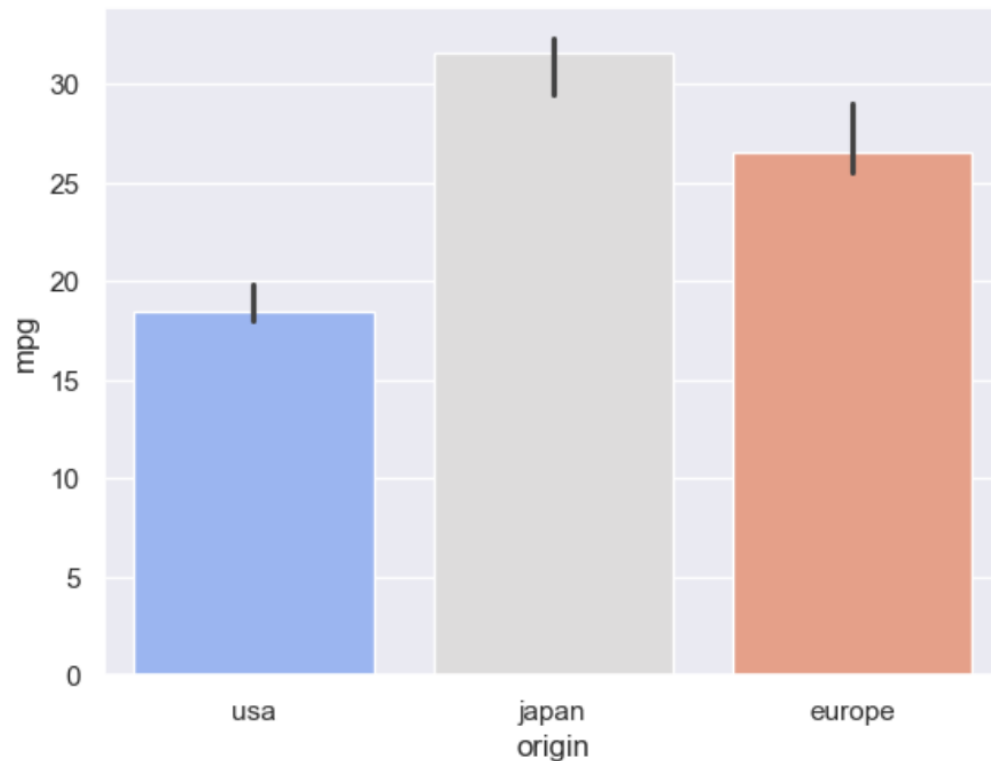
- Una variable **categorica**
- Una variable **numérica**
- **Agregadas** por la media
- **estimator**: cambia la función de agregación

Visualización con seaborn

Seaborn

■ barplot en seaborn

```
• [47]: sns.barplot(x="origin", y="mpg", data=dat, estimator=np.median,  
                  hue="origin", palette="coolwarm")  
plt.show()
```



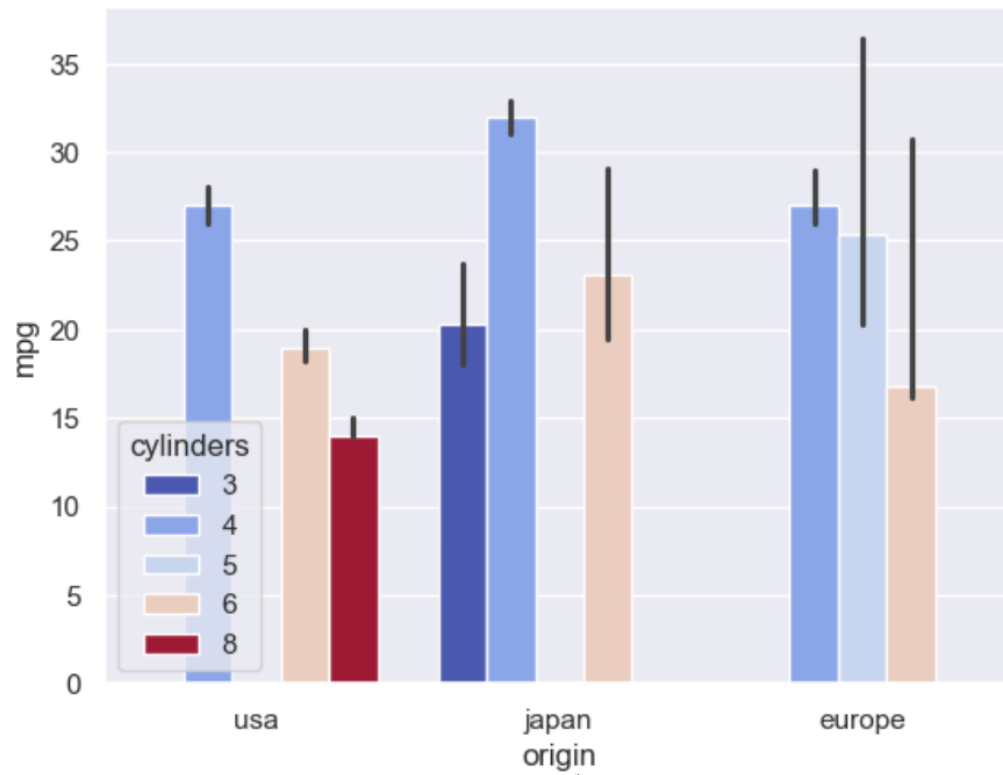
- Una variable **categorica**
- Una variable **numérica**
- **Agregadas** por la media
- **estimator**: cambia la función de agregación

Visualización con seaborn

Seaborn

■ barplot en seaborn

```
[56]: sns.barplot(x="origin", y="mpg", data=dat, estimator=np.median,  
                hue="cylinders", palette="coolwarm")  
plt.show()
```



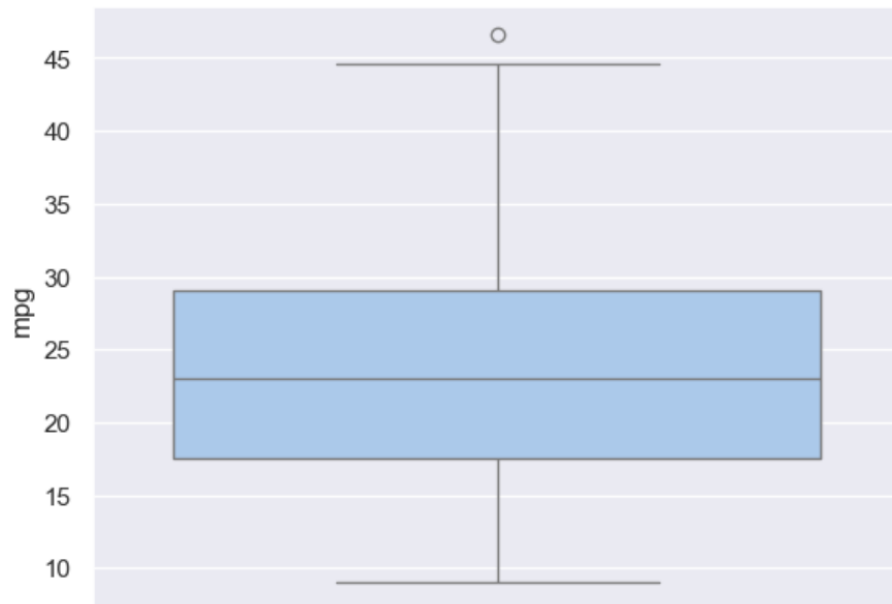
- Una variable **categorica**
- Una variable **numérica**
- **Agregadas** por la media
- **estimator**: cambia la función de agregación

Visualización con seaborn

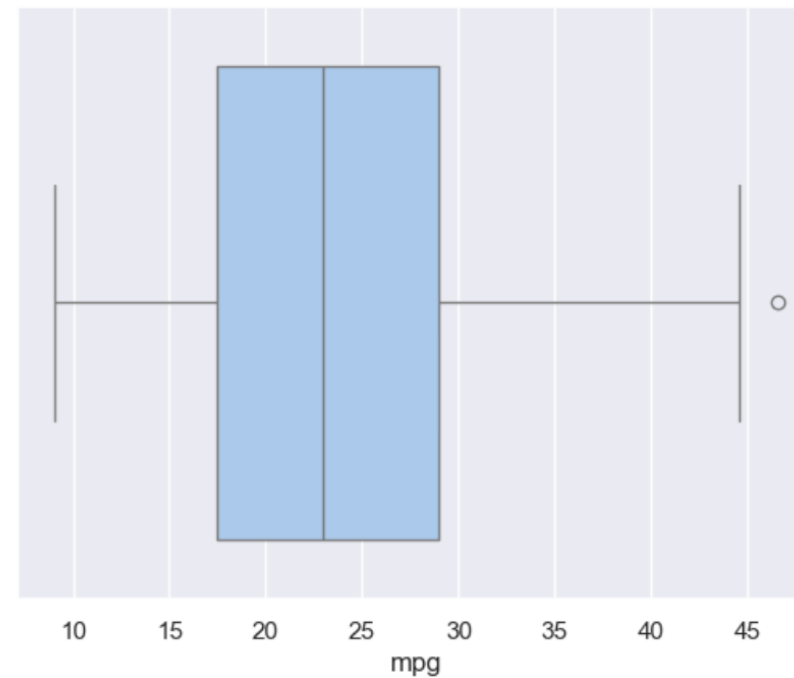
Seaborn

- boxplot en seaborn

```
[58]: sns.boxplot(y="mpg", data=dat)  
plt.show()
```



```
[59]: sns.boxplot(x="mpg", data=dat)  
plt.show()
```

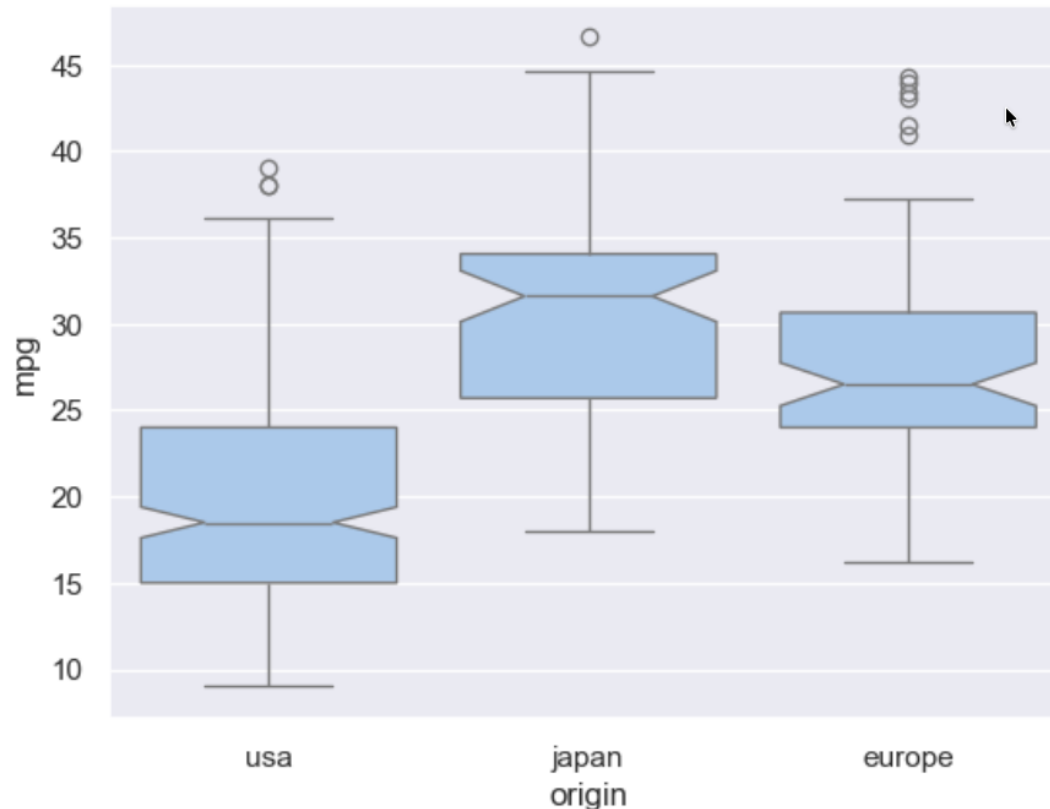


Visualización con seaborn

Seaborn

■ boxplot en seaborn

```
[67]: sns.boxplot(data=dat, x="origin", y="mpg", notch=True)  
plt.show()
```



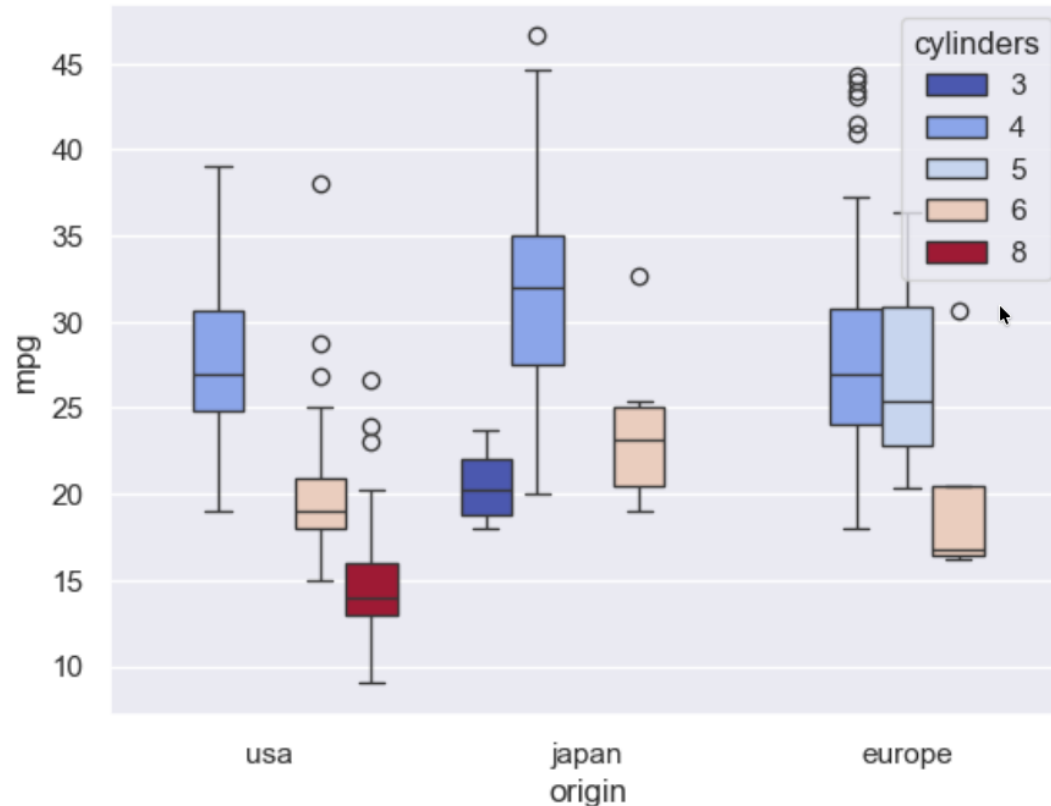
- **notch**: hendidura que aparece en los lados de la caja de un boxplot.
- Representa un **intervalo de confianza** aproximado alrededor de la mediana (habitualmente al 95%).
- Cuanto más **estrecha** es la muesca, más **precisa** es la estimación de la mediana.
- Si las muescas de dos boxplots no se solapan, se interpreta como una **diferencia estadísticamente significativa** entre las medianas de los grupos.

Visualización con seaborn

Seaborn

■ boxplot en seaborn

```
[69]: sns.boxplot(data=dat, x="origin", y="mpg", hue="cylinders", palette="coolwarm")  
plt.show()
```



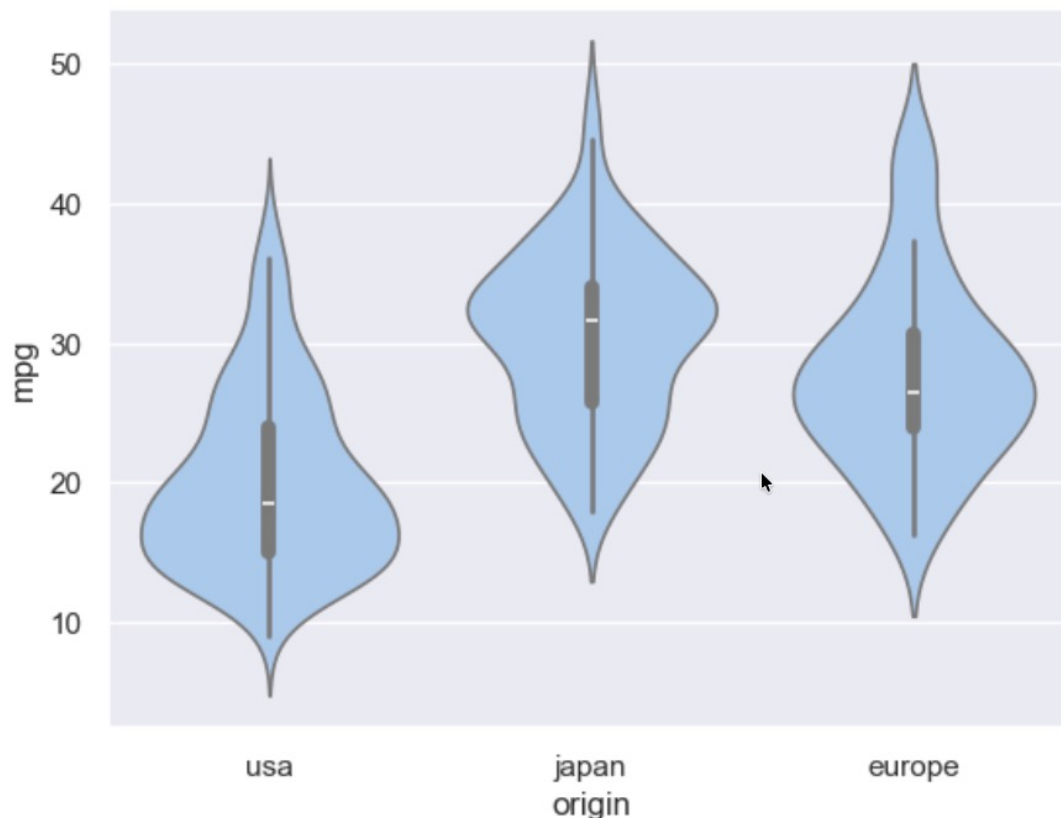
- **notch**: hendidura que aparece en los lados de la caja de un boxplot.
- Representa un **intervalo de confianza** aproximado alrededor de la mediana (habitualmente al 95%).
- Cuanto más **estrecha** es la muesca, más **precisa** es la estimación de la mediana.
- Si las muescas de dos boxplots no se solapan, se interpreta como una **diferencia estadísticamente significativa** entre las medianas de los grupos.

Visualización con seaborn

Seaborn

■ violinplot en seaborn

```
[71]: sns.violinplot(data=dat, x="origin", y="mpg")  
plt.show()
```



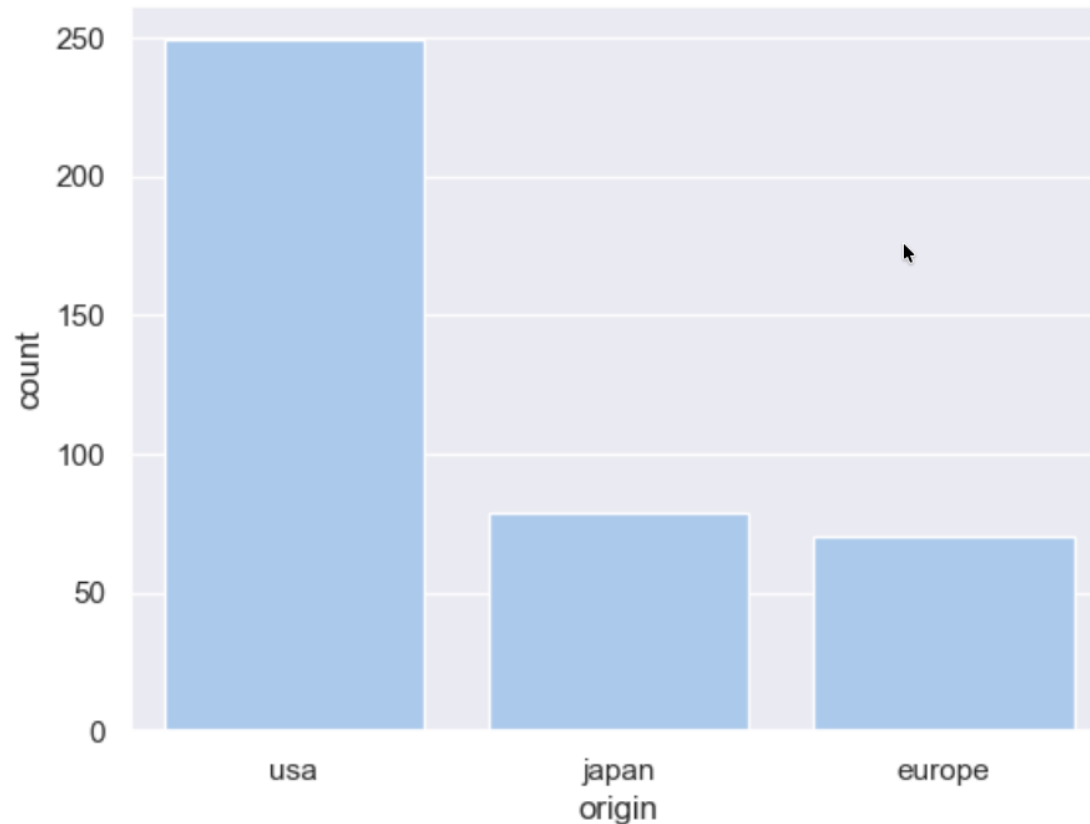
- Combina características de un **boxplot** y de un gráfico de **densidad** (KDE).
- Muestra la distribución de los datos a través de su densidad estimada (curva suavizada reflejada simétricamente → forma de “violín”).
- En el **centro** suele incluir un **boxplot** (mediana, cuartiles, bigotes).
- Permite ver no solo medidas resumidas (como el boxplot), sino también la **forma completa de la distribución** (asimetrías, multimodalidad, colas largas).

Visualización con seaborn

Seaborn

■ countplot en seaborn

```
[76]: sns.countplot(x="origin", data=dat)  
plt.show()
```



- Muestra el **recuento** (frecuencia absoluta) de cada categoría de una variable **cualitativa**.
- Es básicamente un **histograma para variables categóricas**.

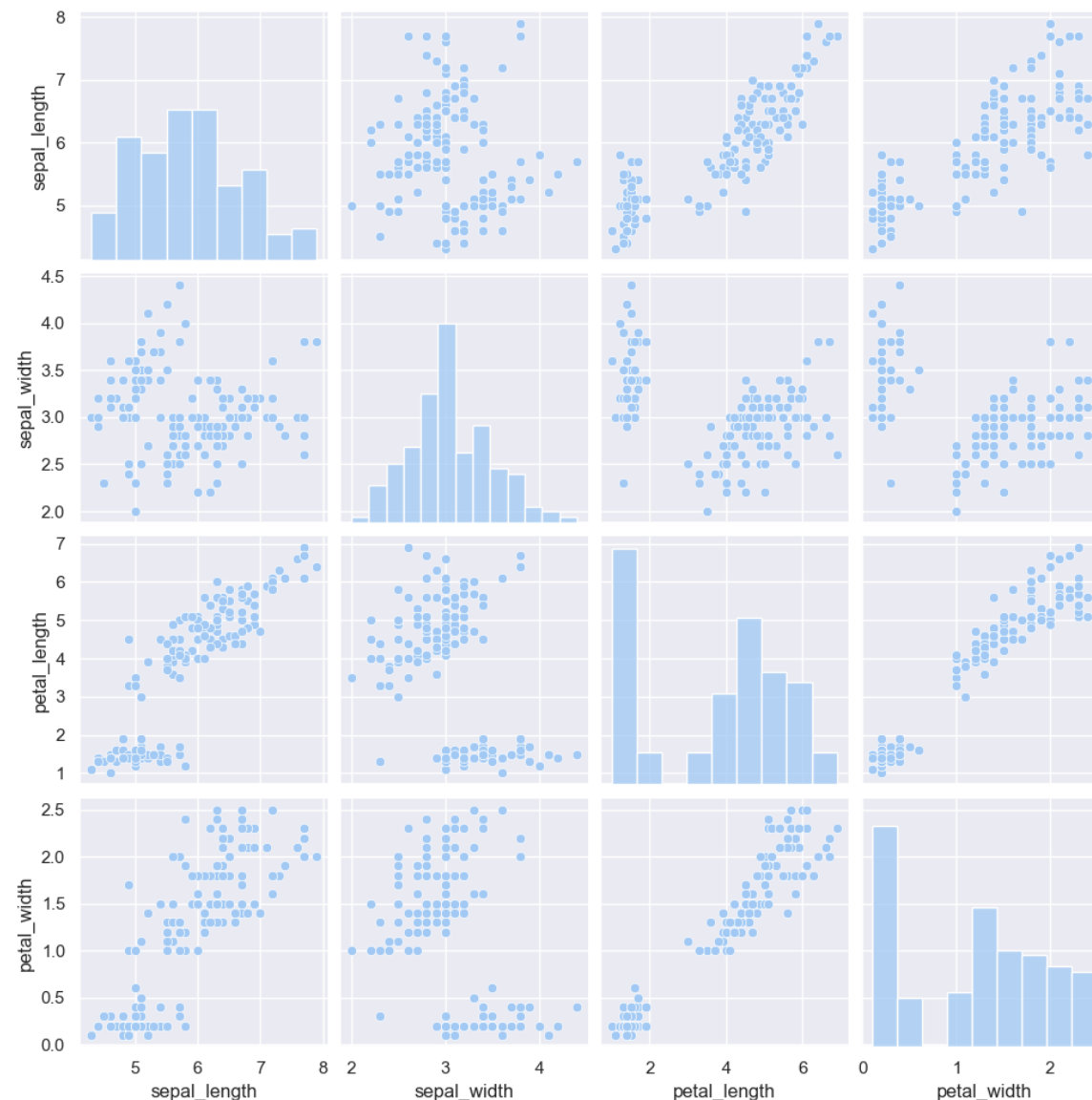
Visualización con seaborn

Seaborn

- pairplot en seaborn
- Muestra de forma automática **todas las combinaciones posibles** de pares de variables en un dataset, junto con sus distribuciones individuales.
- **Bivariado**: cada celda fuera de la diagonal es un gráfico (normalmente scatter) que representa la relación entre dos variables.
- **Univariado**: en la diagonal aparecen histogramas o KDEs que muestran la distribución de cada variable.
- Es muy útil para **exploración inicial de datos multivariados**, porque permite detectar correlaciones, patrones y posibles outliers de un vistazo.

```
[78]: dat = sns.load_dataset("iris")
sns.pairplot(data=dat)

[78]: <seaborn.axisgrid.PairGrid at 0x19aa8ebd0>
```



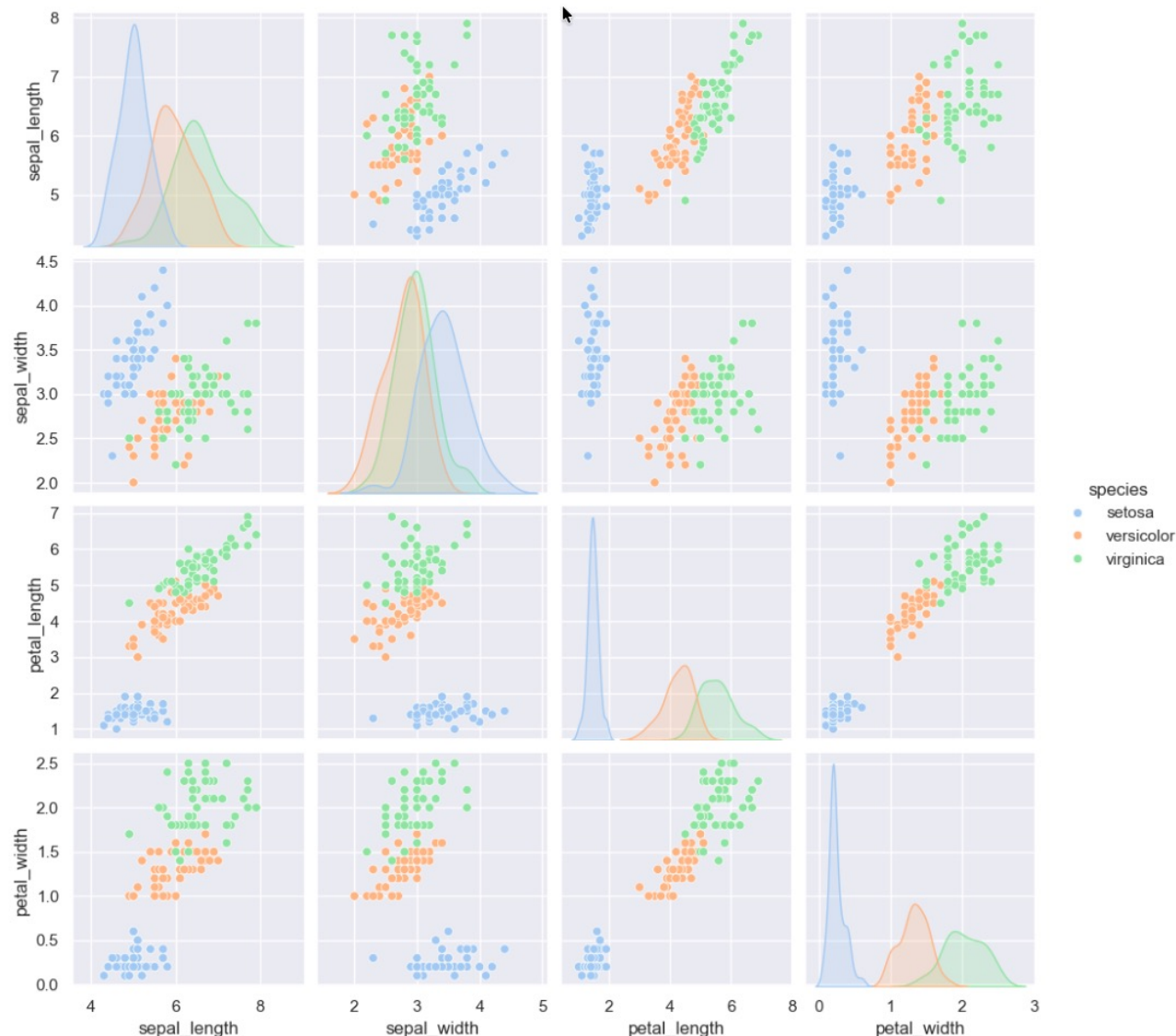
Visualización con seaborn

Seaborn

- pairplot en seaborn
- Muestra de forma automática **todas las combinaciones posibles** de pares de variables en un dataset, junto con sus distribuciones individuales.
- **Bivariado**: cada celda fuera de la diagonal es un gráfico (normalmente scatter) que representa la relación entre dos variables.
- **Univariado**: en la diagonal aparecen histogramas o KDEs que muestran la distribución de cada variable.
- Es muy útil para **exploración inicial de datos multivariados**, porque permite detectar correlaciones, patrones y posibles outliers de un vistazo.

```
[79]: sns.pairplot(data=dat, hue="species")
```

```
[79]: <seaborn.axisgrid.PairGrid at 0x19ea7a510>
```

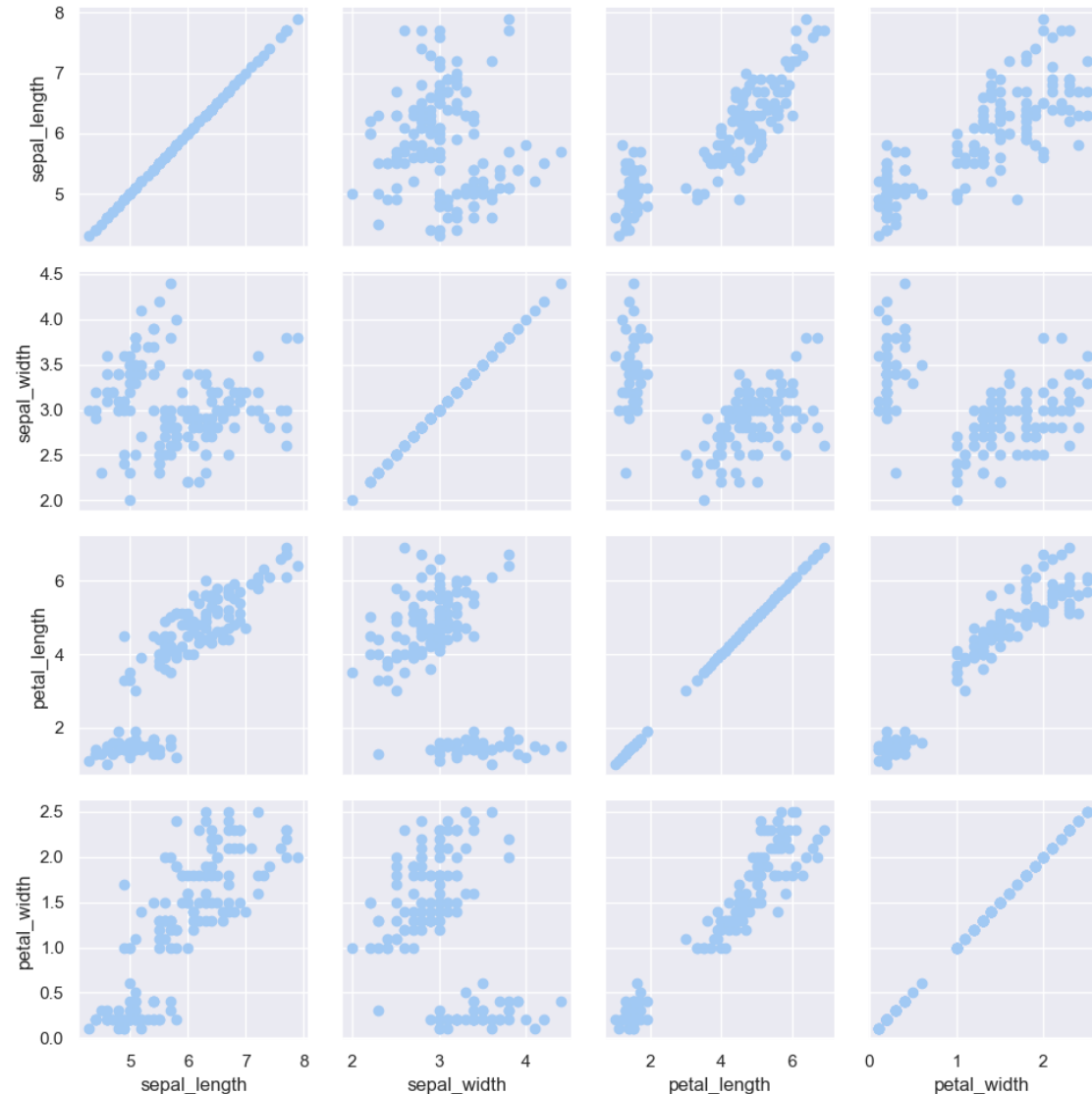


Visualización con seaborn

Seaborn

- PairGrid en seaborn
- Un **PairGrid** permite construir de forma flexible una **matriz de gráficos** mostrando todas las relaciones posibles entre varias variables.
- Es la base en la que se construye pairplot, pero da mucho más control.
- Divide la figura en:
- **Diagonal** → suele mostrar distribuciones univariadas (histograma, KDE...).
- **Parte inferior y superior** → gráficos bivariados entre pares de variables.
- Permite elegir **diferentes tipos de gráficos** para cada parte de la matriz (scatter, kde, reg, hist, etc.).
- Se puede usar **map_lower()**, **map_upper()** y **map_diag()** para especificar qué tipo de gráfico colocar en cada zona.

```
[80]: g = sns.PairGrid(dat)
      g.map(plt.scatter)
      plt.show()
```

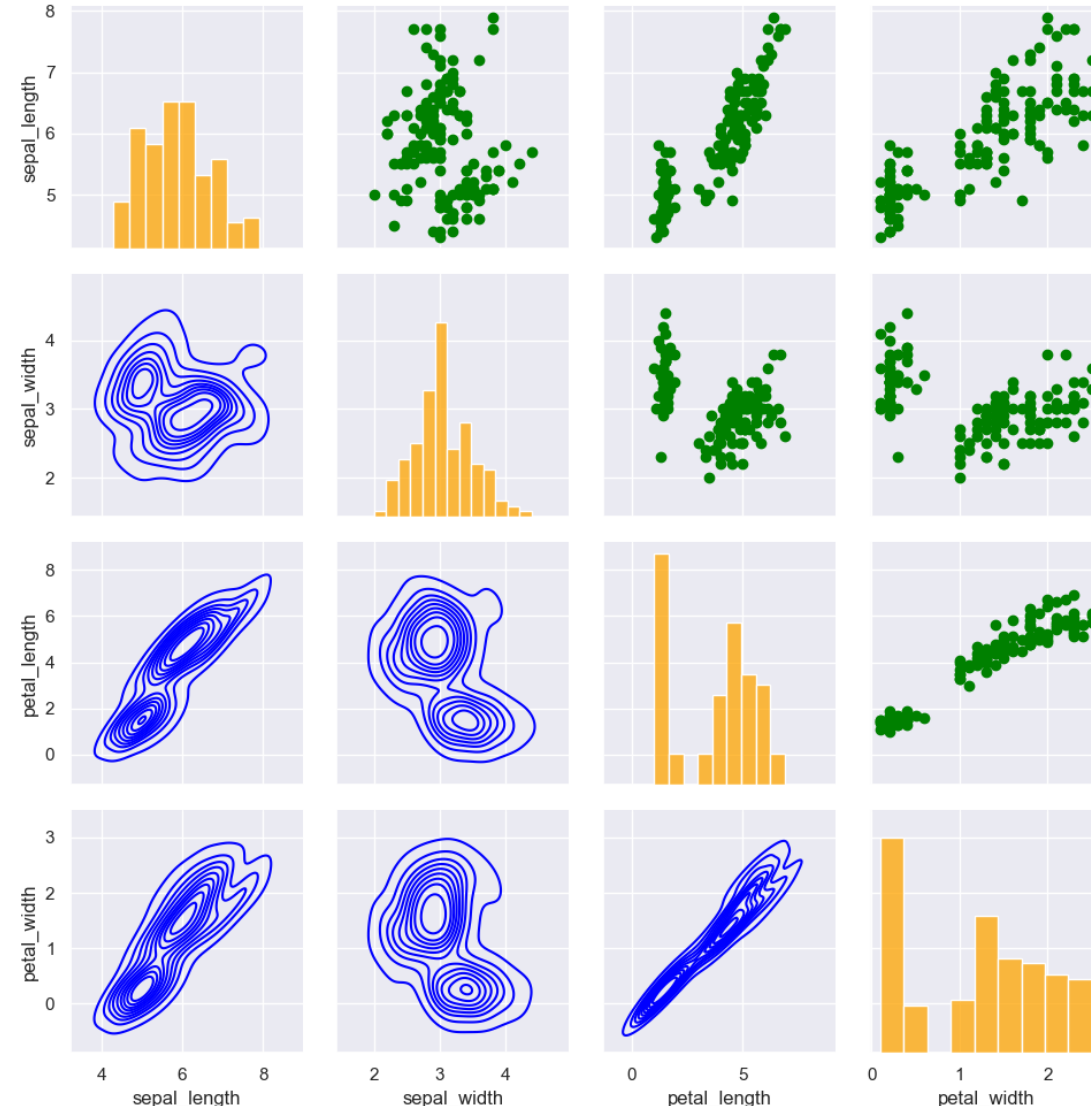


Visualización con seaborn

Seaborn

- PairGrid en seaborn
- Un **PairGrid** permite construir de forma flexible una **matriz de gráficos** mostrando todas las relaciones posibles entre varias variables.
- Es la base en la que se construye pairplot, pero da mucho más control.
- Divide la figura en:
- **Diagonal** → suele mostrar distribuciones univariadas (histograma, KDE...).
- **Parte inferior y superior** → gráficos bivariados entre pares de variables.
- Permite elegir **diferentes tipos de gráficos** para cada parte de la matriz (scatter, kde, reg, hist, etc.).
- Se puede usar **map_lower()**, **map_upper()** y **map_diag()** para especificar qué tipo de gráfico colocar en cada zona.

```
[82]: g = sns.PairGrid(dat)
      g.map_diag(sns.histplot, color="orange")
      g.map_upper(plt.scatter, color="green")
      g.map_lower(sns.kdeplot, color="blue")
      plt.show()
```



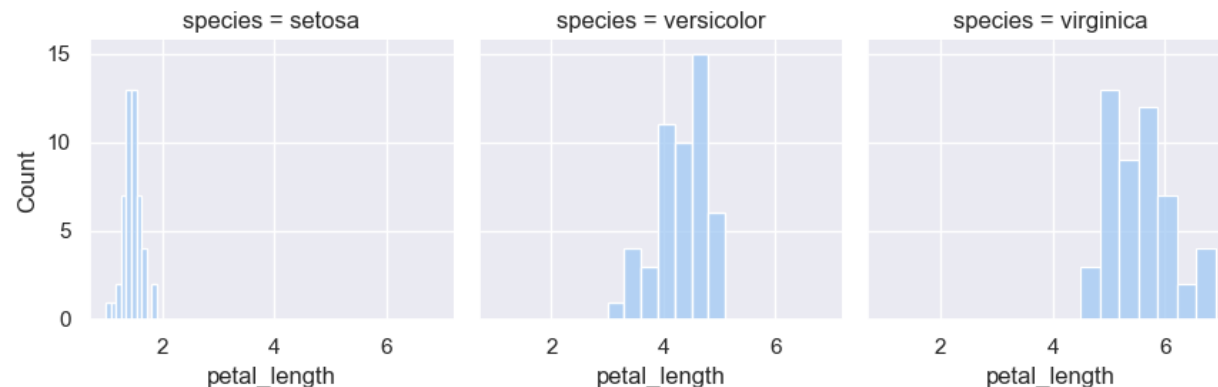
Visualización con seaborn

Seaborn

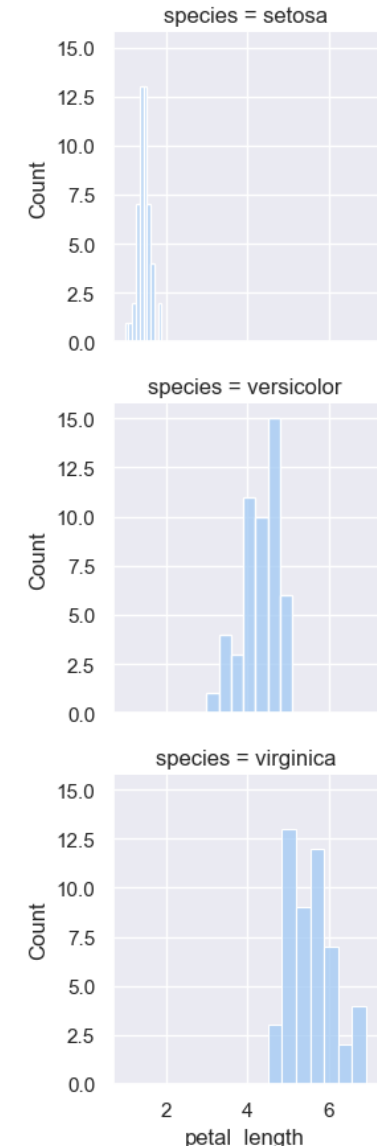
■ FacetGrid en seaborn

- Estructura que permite crear una **cuadrícula de subgráficos** (facetas) en función de una o más variables categóricas.
- Cada subgráfico (facet) muestra un subconjunto de los datos.
- Sirve para **comparar** fácilmente cómo cambia una relación o distribución entre diferentes categorías.
- Puedes dividir la cuadrícula en **filas** (row), **columnas** (col) y también usar **colores** (hue) para añadir una tercera dimensión categórica.

```
[83]: g = sns.FacetGrid(data=dat, col="species")
      g.map(sns.histplot, "petal_length")
      plt.show()
```



```
[84]: g = sns.FacetGrid(data=dat, row="species")
      g.map(sns.histplot, "petal_length")
      plt.show()
```



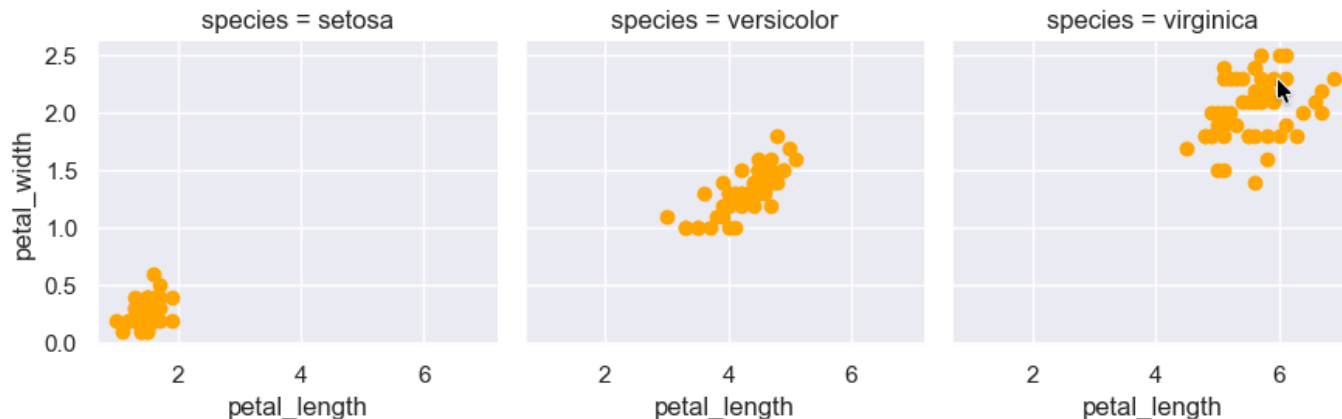
Visualización con seaborn

Seaborn

■ FacetGrid en seaborn

- Estructura que permite crear una **cuadrícula de subgráficos** (facetas) en función de una o más variables categóricas.
- Cada subgráfico (facet) muestra un subconjunto de los datos.
- Sirve para **comparar** fácilmente cómo cambia una relación o distribución entre diferentes categorías.
- Puedes dividir la cuadrícula en **filas** (row), **columnas** (col) y también usar **colores** (hue) para añadir una tercera dimensión categórica.

```
[85]: g = sns.FacetGrid(data=dat, col="species")
      g.map(plt.scatter, "petal_length", "petal_width", color="orange")
      plt.show()
```



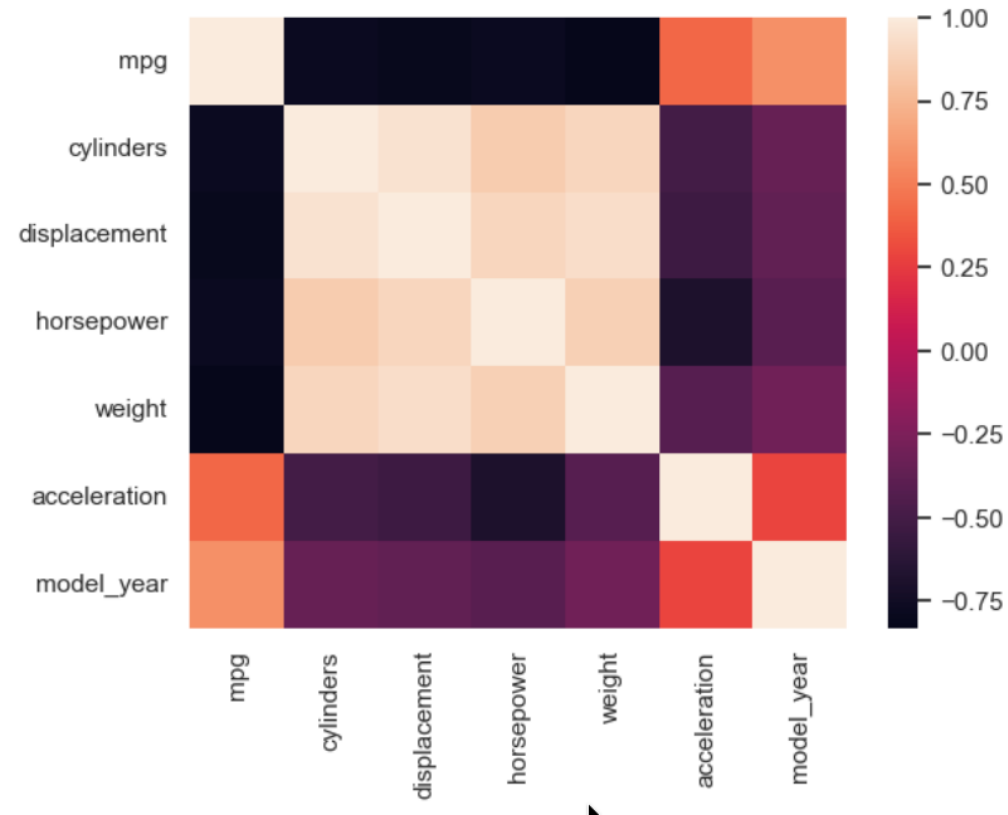
Visualización con seaborn

Seaborn

■ heatmap en seaborn

- Representación gráfica en forma de **matriz de colores**.
- Filas y columnas representan **variables o categorías**.
- El **color** de cada celda indica la **magnitud** del valor en esa posición (por ejemplo, intensidad, frecuencia correlación).
- Colores más claros u oscuros (o distintos tonos) permiten identificar **patrones** de manera visual e inmediata.

```
[87]: dat = sns.load_dataset("mpg")
      dat.drop(columns=["origin", "name"], inplace=True)
      sns.heatmap(dat.corr())
      plt.show()
```



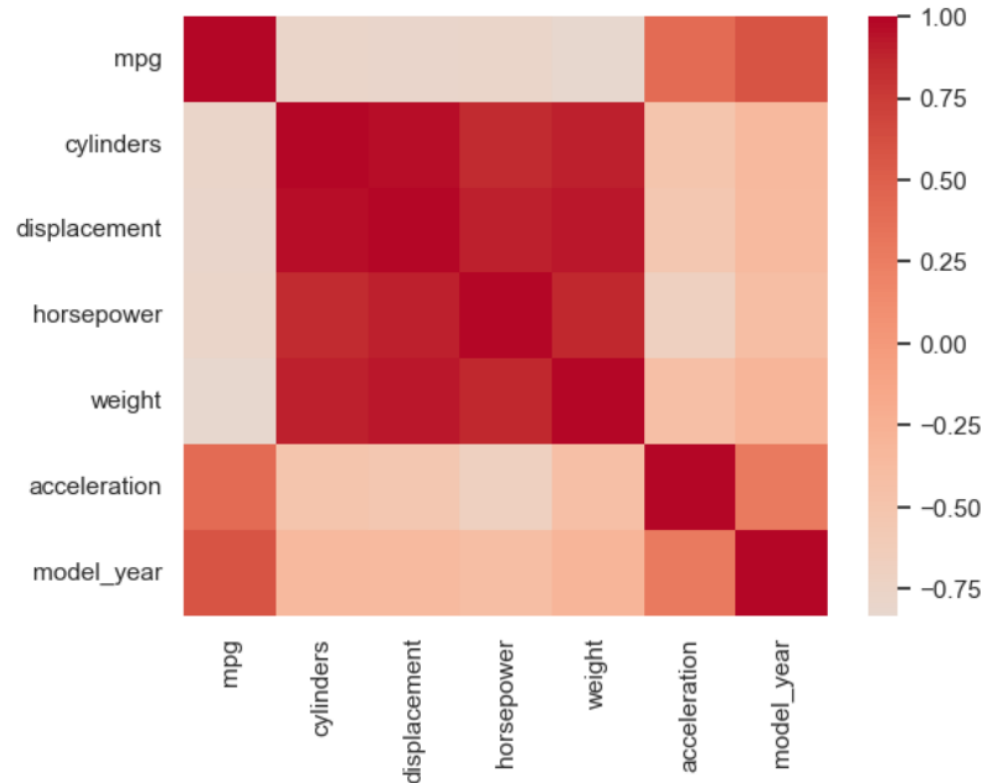
Visualización con seaborn

Seaborn

■ heatmap en seaborn

- Representación gráfica en forma de **matriz de colores**.
- Filas y columnas representan **variables o categorías**.
- El **color** de cada celda indica la **magnitud** del valor en esa posición (por ejemplo, intensidad, frecuencia o correlación).
- Colores más claros u oscuros (o distintos tonos) permiten identificar **patrones** de manera visual e inmediata.
- **center**: valor en el que centrar el mapa de colores.

```
[88]: dat = sns.load_dataset("mpg")
dat.drop(columns=["origin", "name"], inplace=True)
sns.heatmap(dat.corr(), cmap="coolwarm", center=-1)
plt.show()
```



Conclusiones

Qué hemos aprendido

- Para entender los datos es importante visualizarlos.
- Matplotlib nos permite visualizar de forma muy sencilla todo tipo de figuras y gráficos.
- Pandas se integra muy bien con matplotlib.
- Seaborn ofrece una interfaz mucho más sencilla para un amplio conjunto de figuras y gráficos.